



Universidade Federal do ABC
Centro de Matemática, Computação e Cognição

Heap Sort

Ordenação Heaps binários

Monael Pinheiro Ribeiro, D.Sc.

Heap Sort

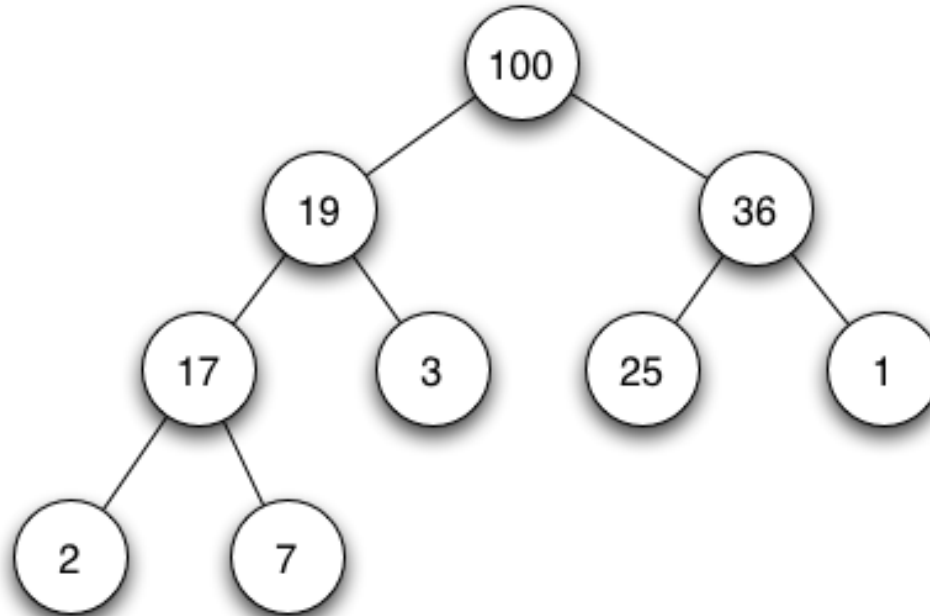
- É um método de ordenação que o faz através de sucessivas seleções do elemento correto.
- Utiliza um heap binário para manter os elementos.
 - Heap binário é uma árvore binária mantida na forma de vetor.
 - O heap é gerado e mantido no próprio vetor a ser ordenado no segmento não ordenado.
- Prós: É mais estável que o Quicksort.
- Contras: Construir o heap consome muita memória.

Heap Binário

Um heap-binário é uma árvore binária por níveis, onde o nó pai de uma subárvore sempre é maior ou igual os seus descendentes

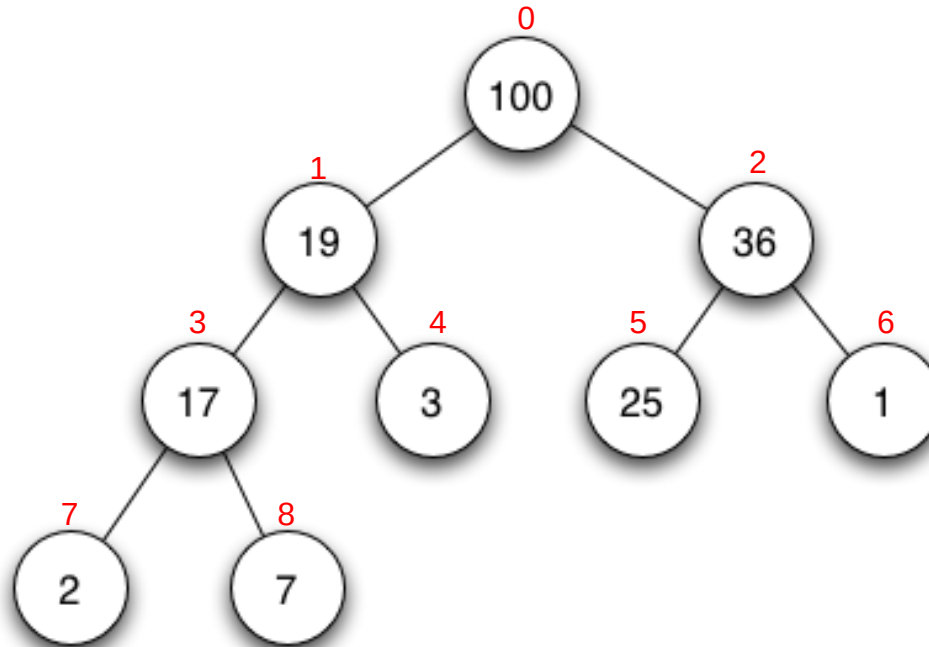
Heap Binário

Um heap-binário é uma árvore binária por níveis, onde o nó pai de uma subárvore sempre é maior ou igual os seus descendentes



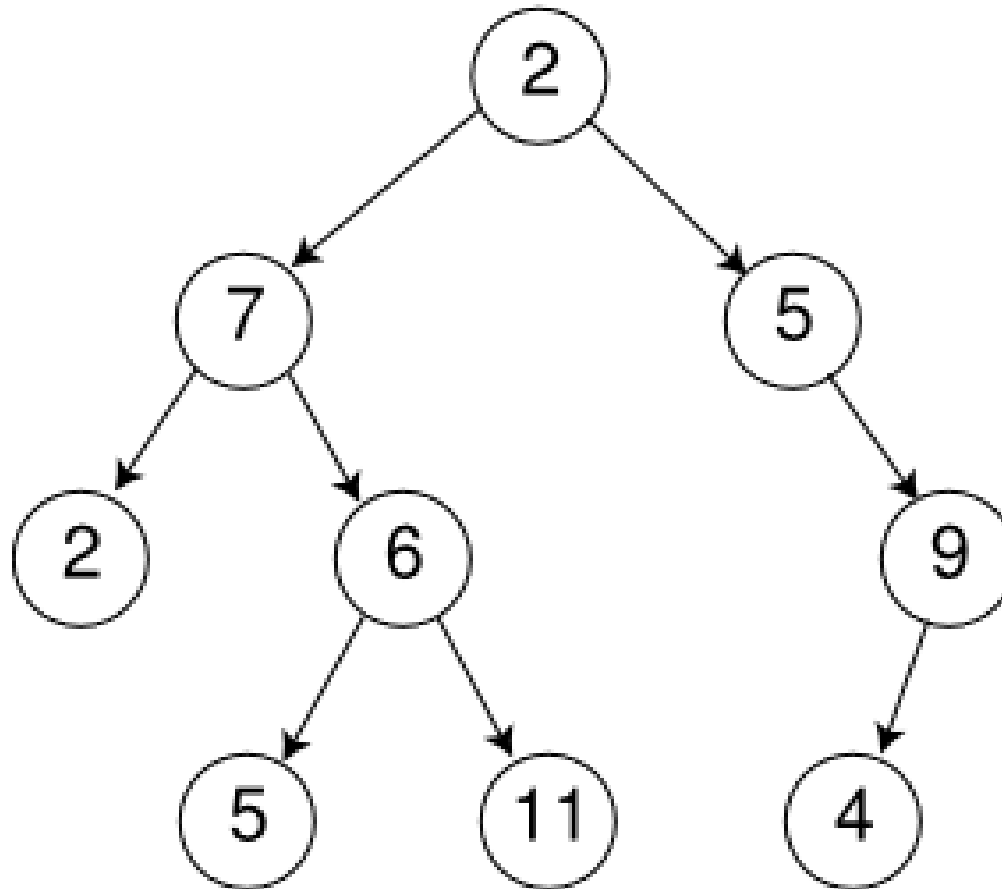
Heap Binário

Um heap-binário é uma árvore binária por níveis, onde o nó pai de uma subárvore sempre é maior ou igual os seus descendentes

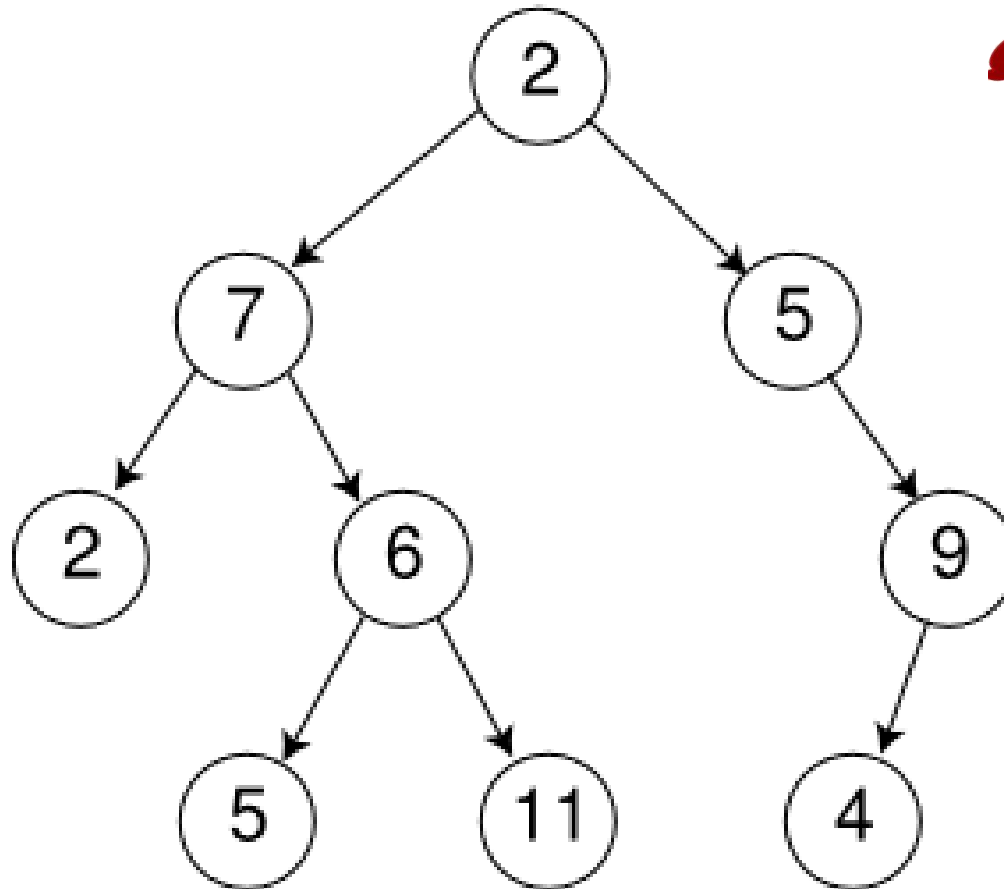


0	1	2	3	4	5	6	7	8
100	19	36	17	3	25	1	2	7

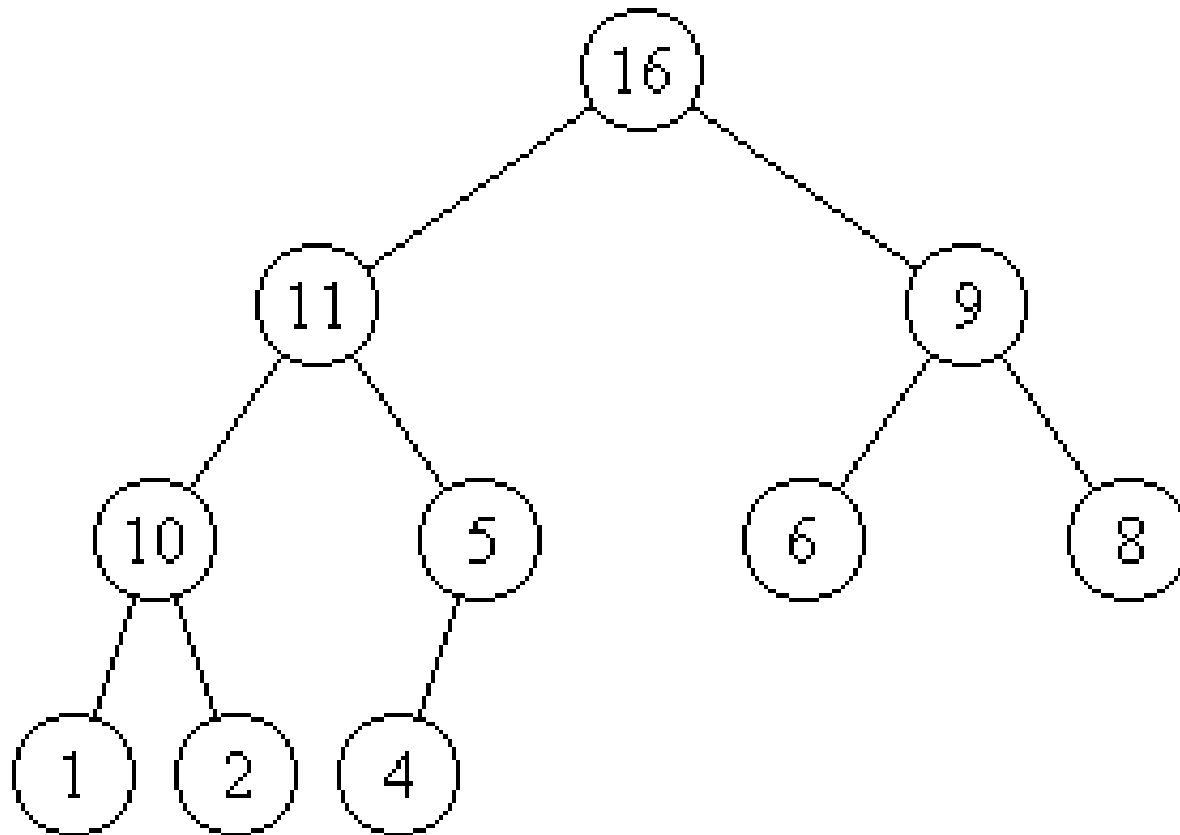
É Heap?



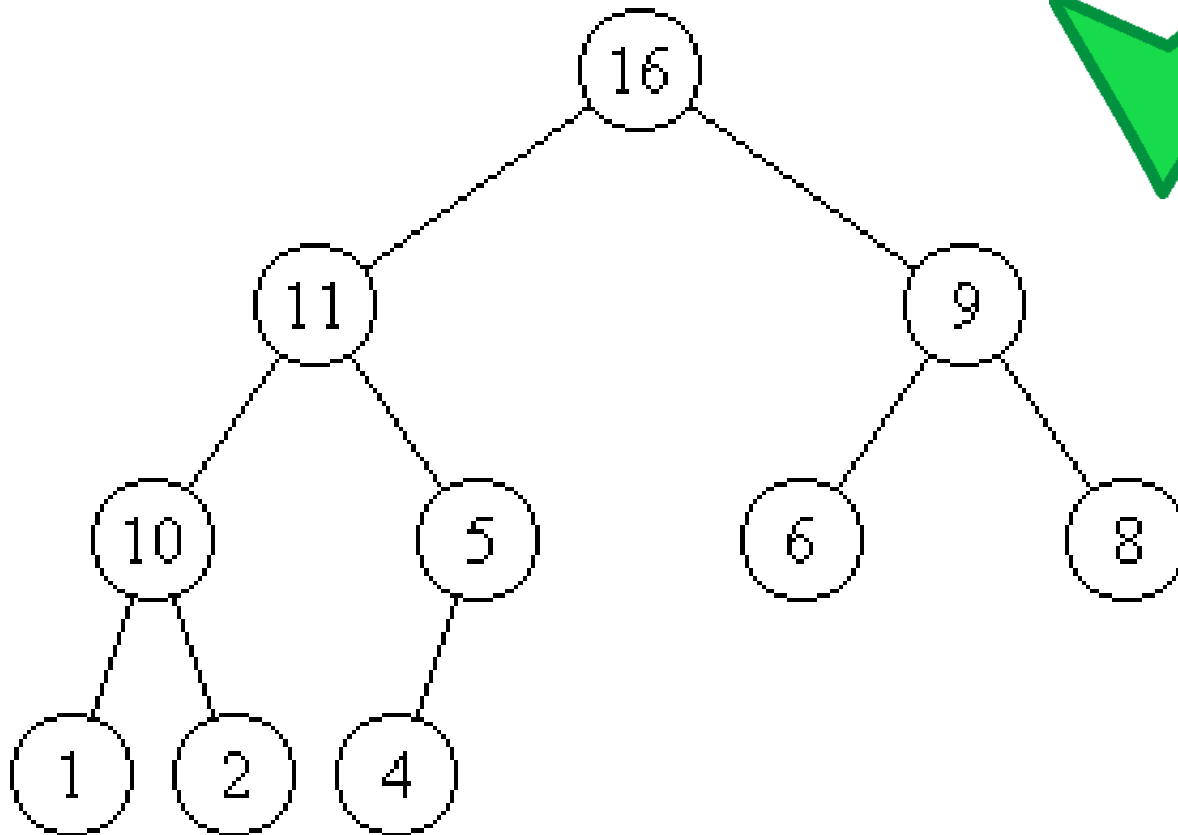
É Heap?



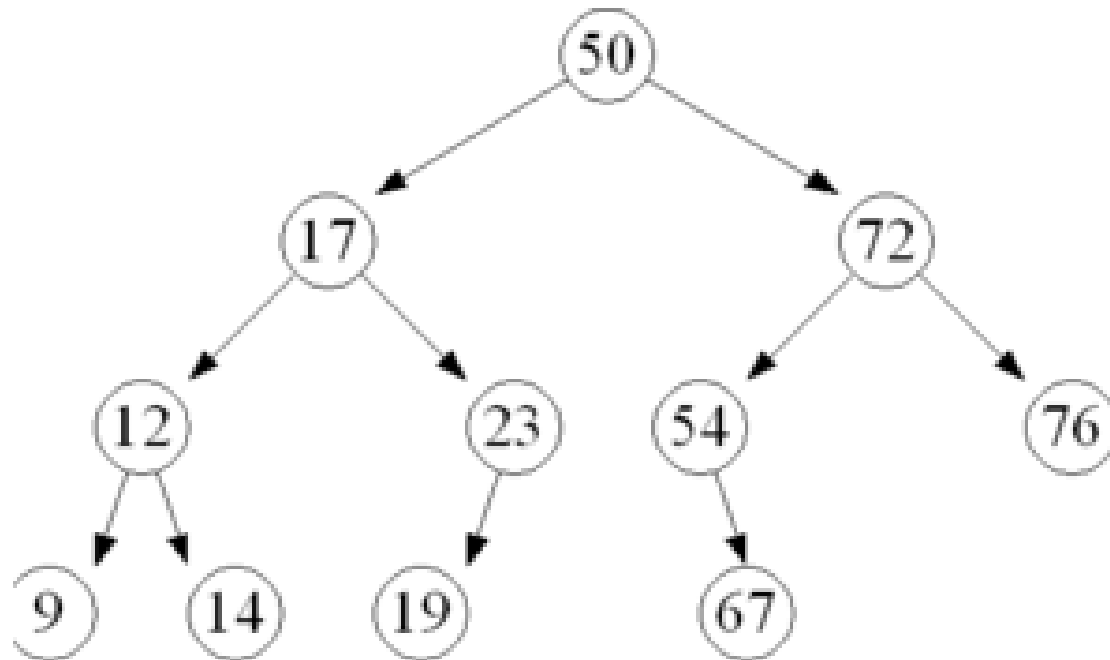
É Heap?



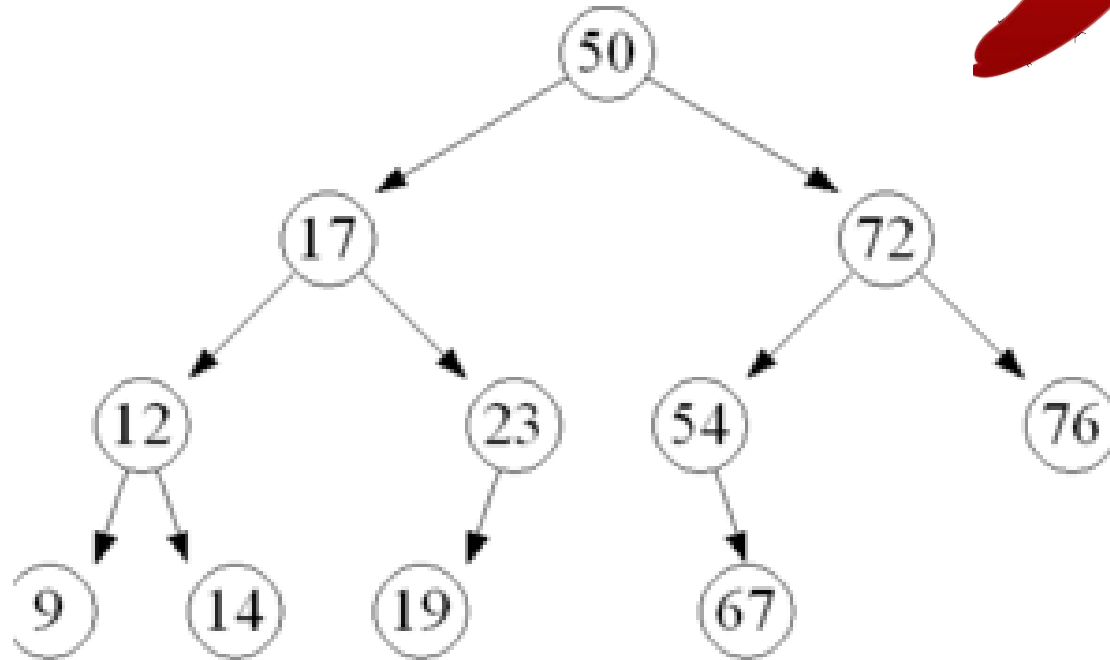
É Heap?



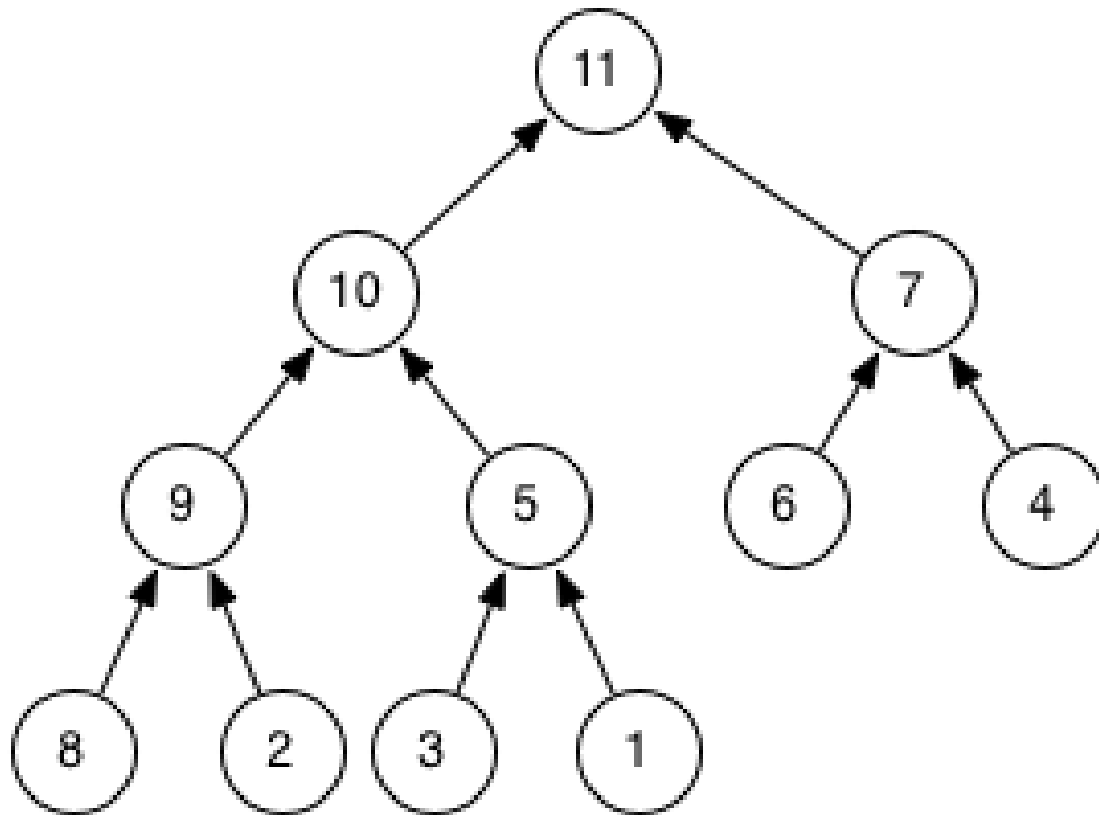
É Heap?



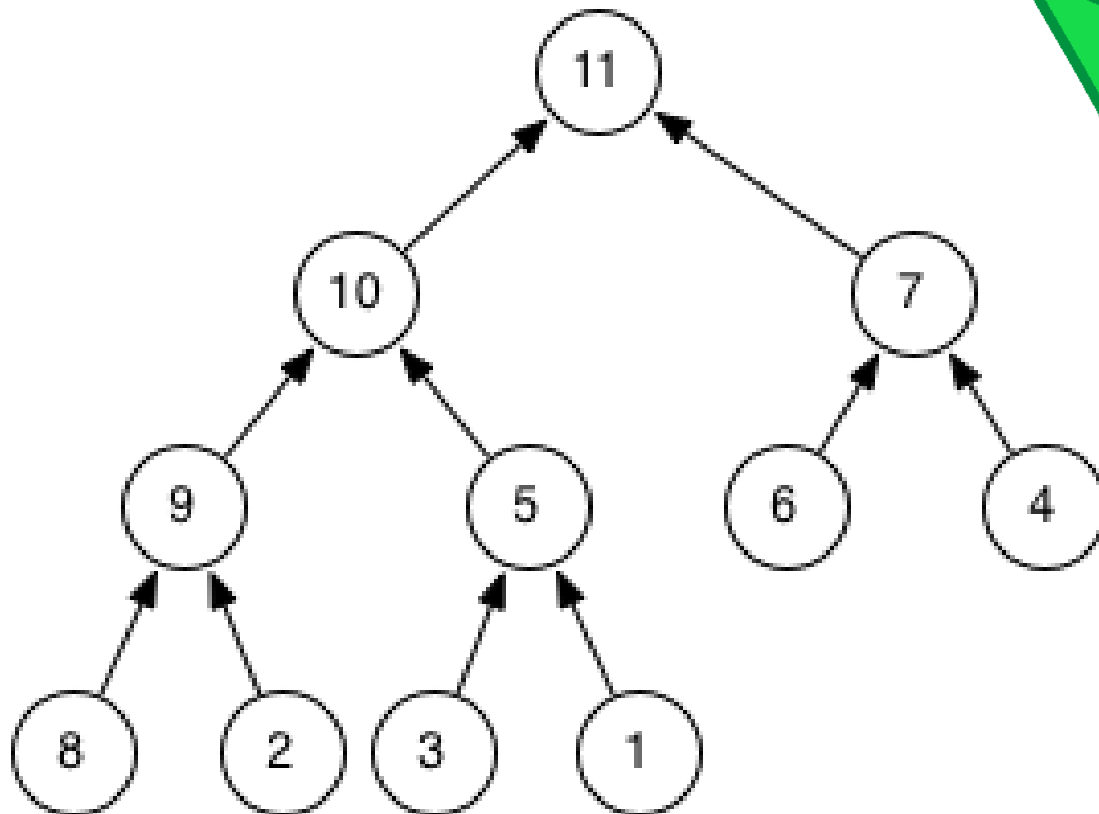
É Heap?



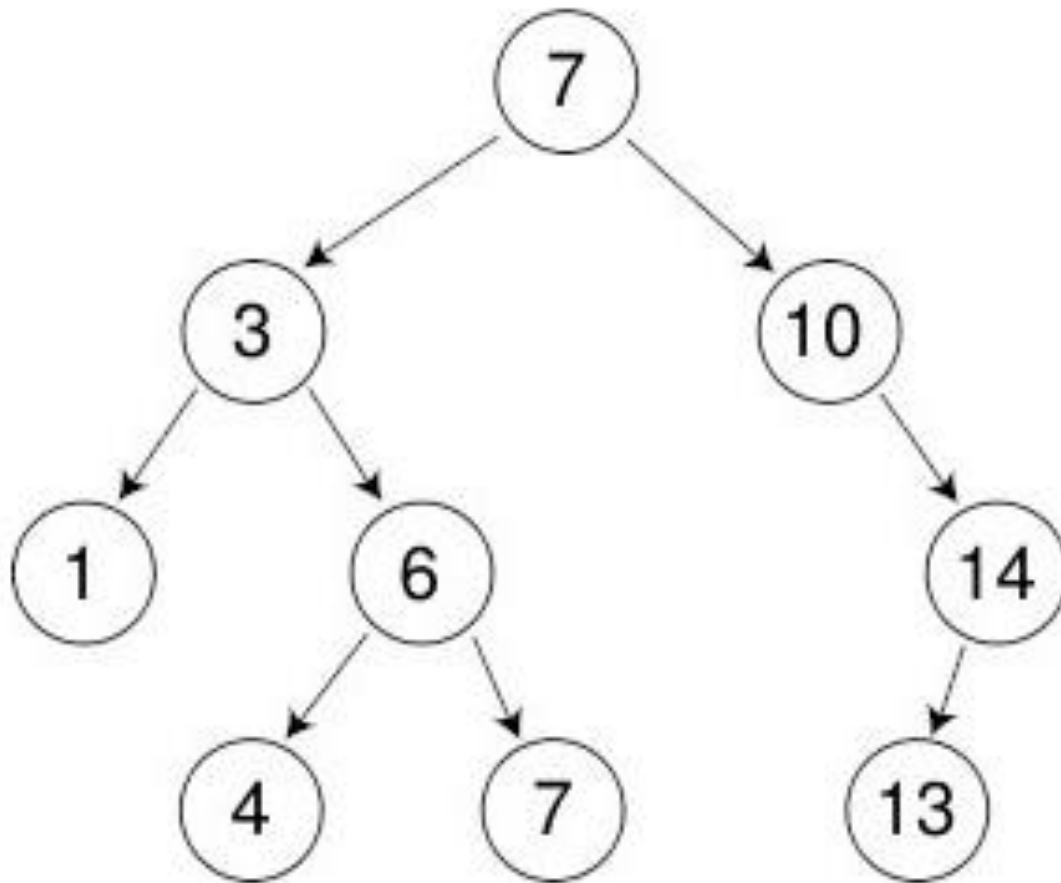
É Heap?



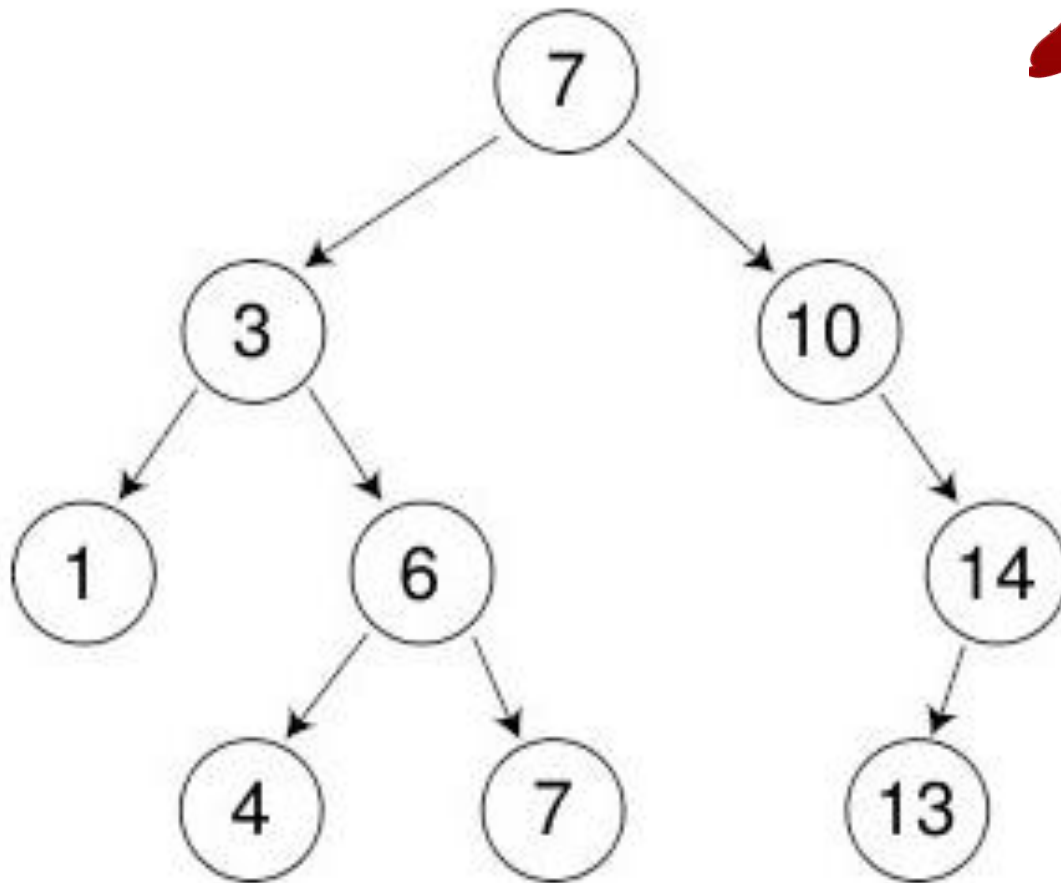
É Heap?



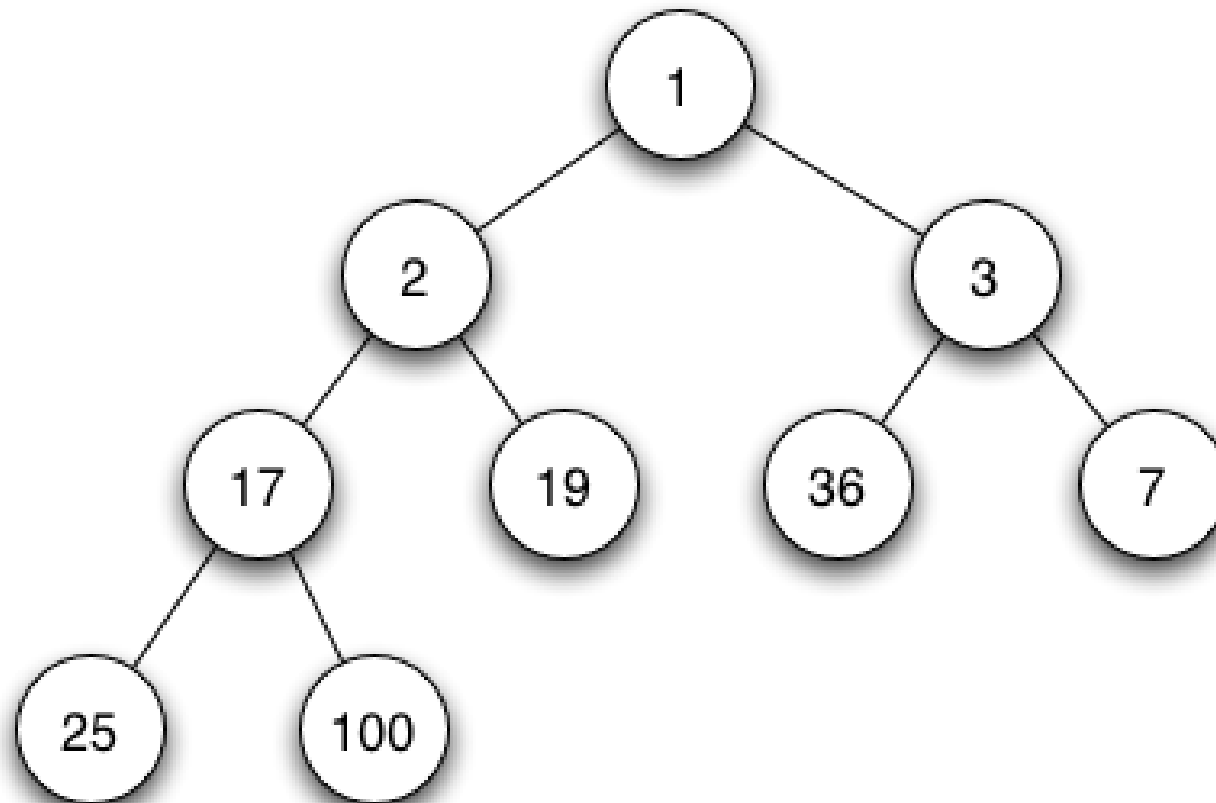
É Heap?



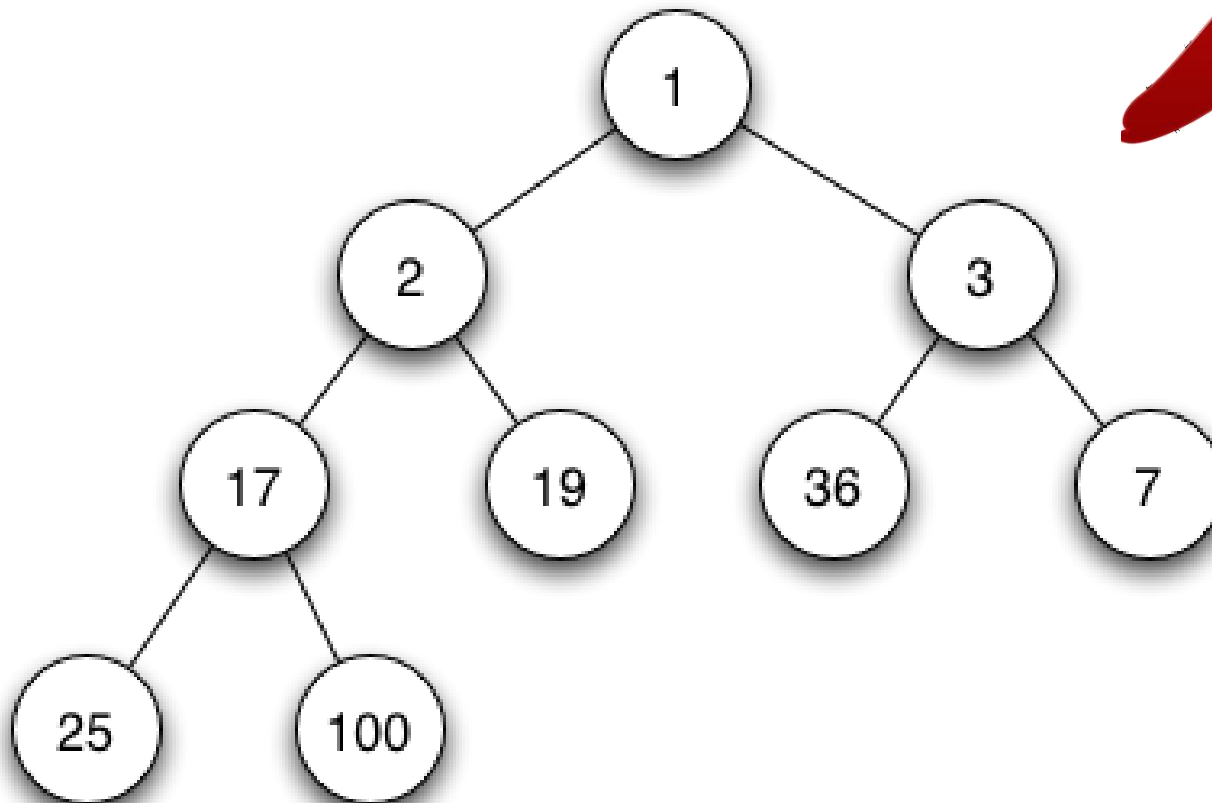
É Heap?



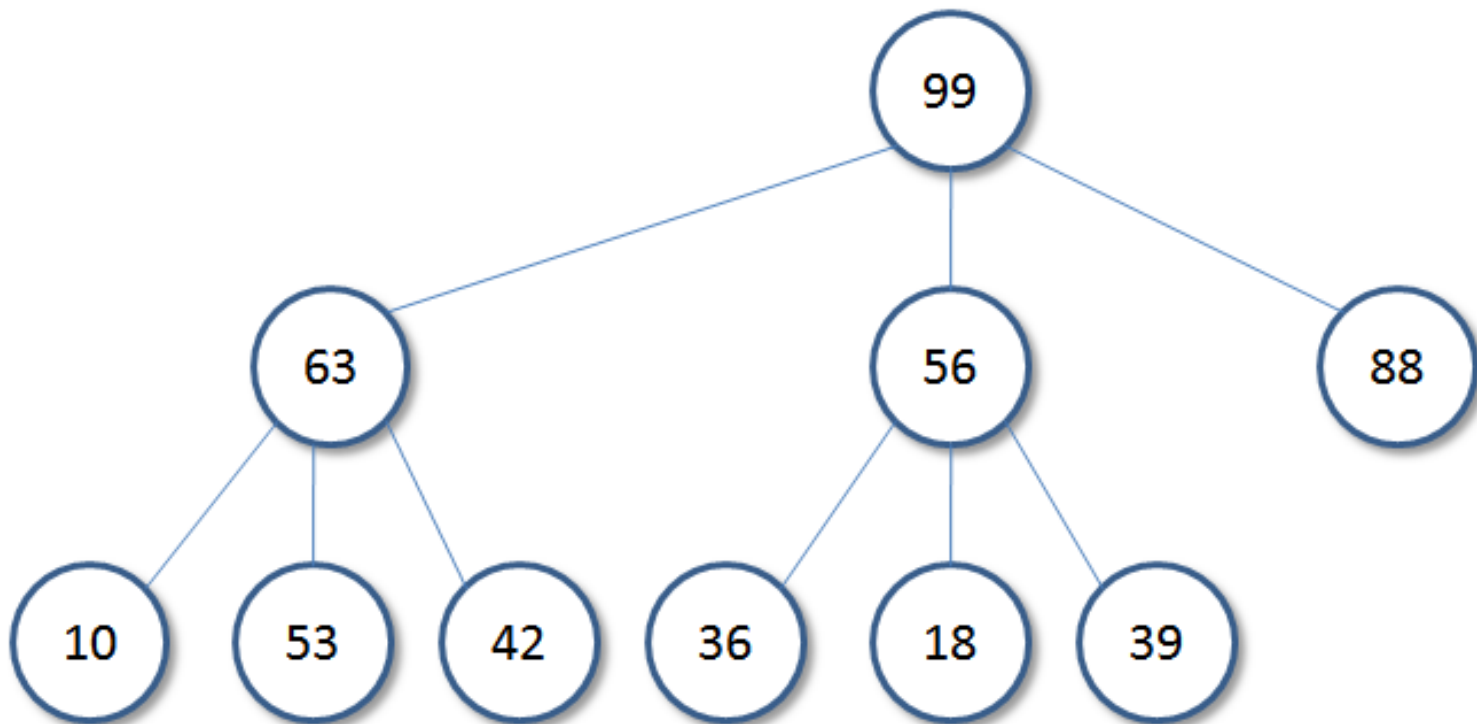
É Heap?



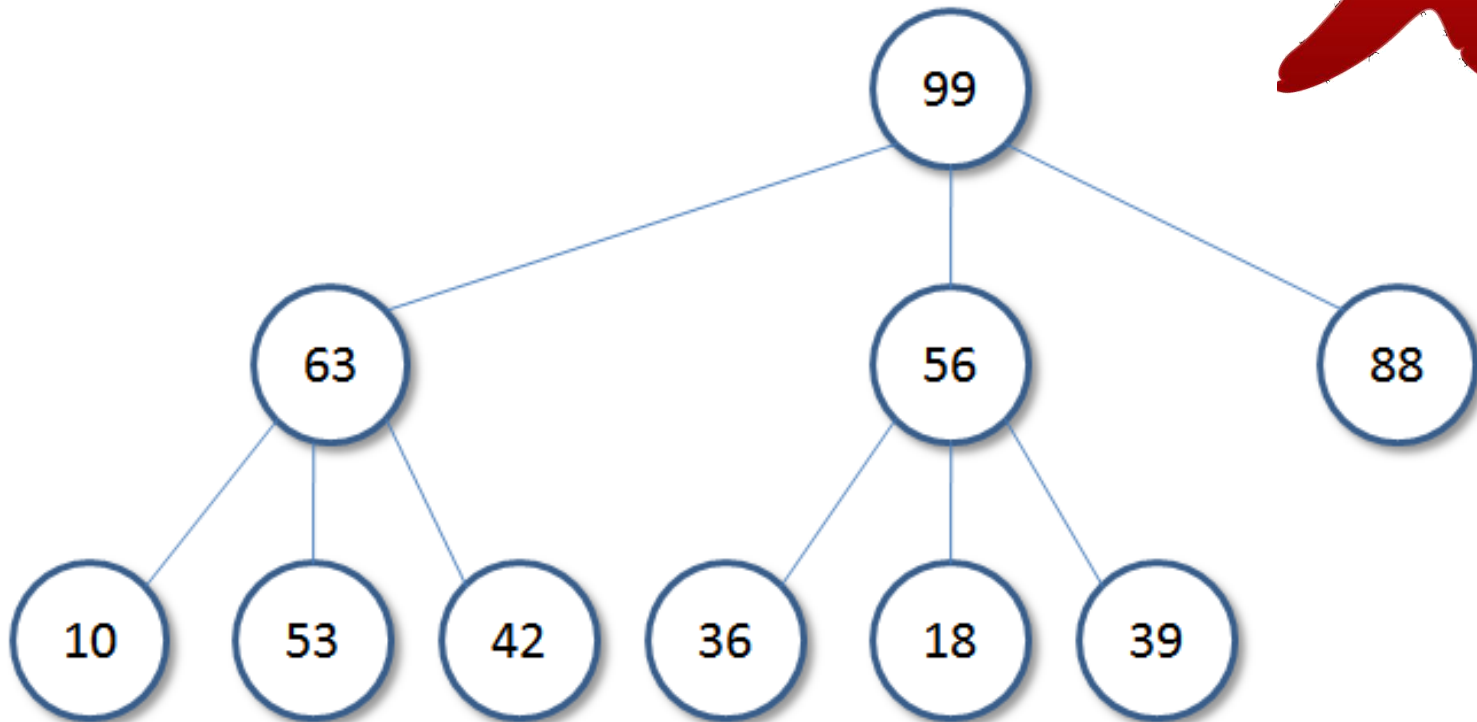
É Heap?



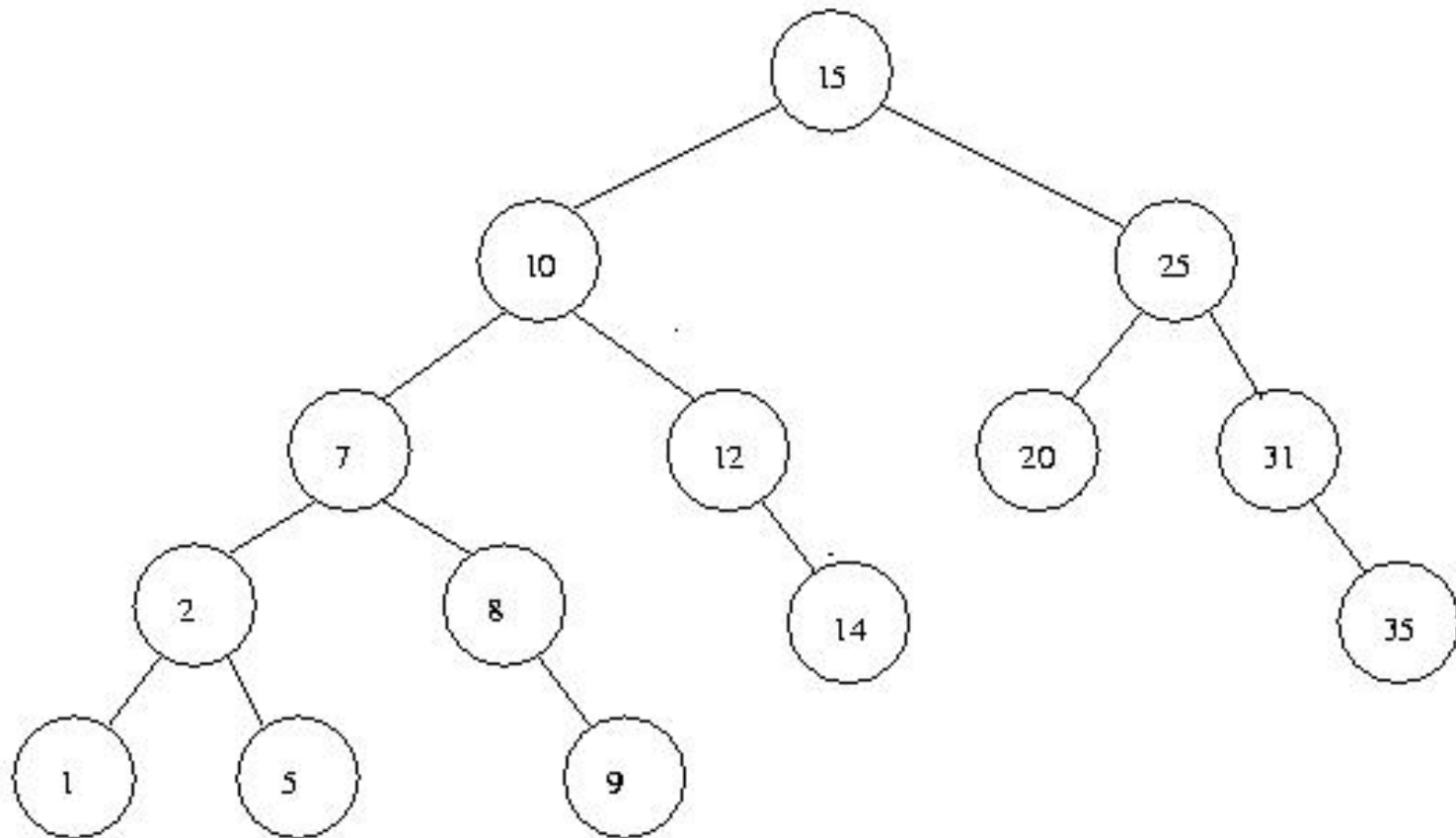
É Heap?



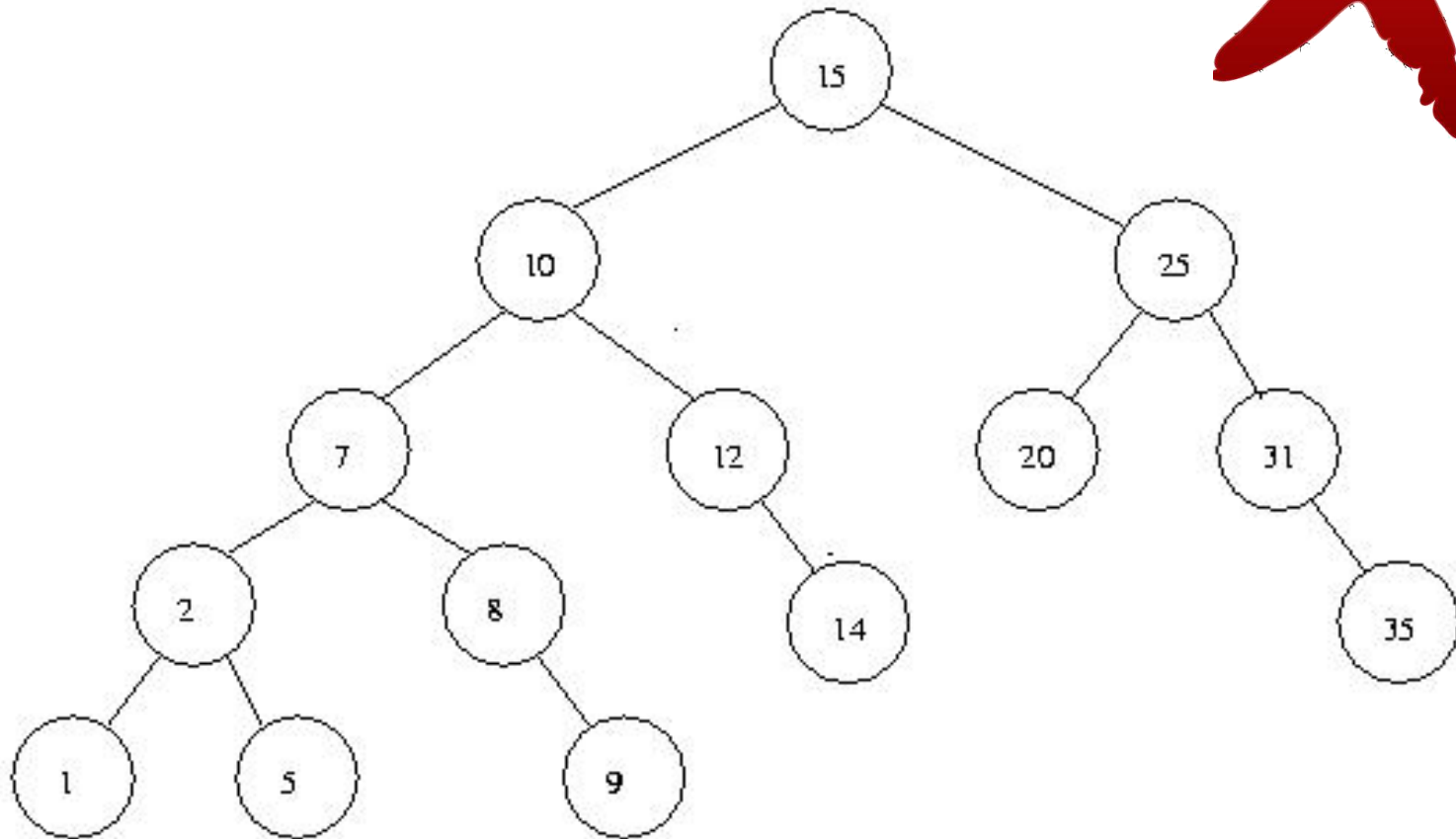
É Heap?



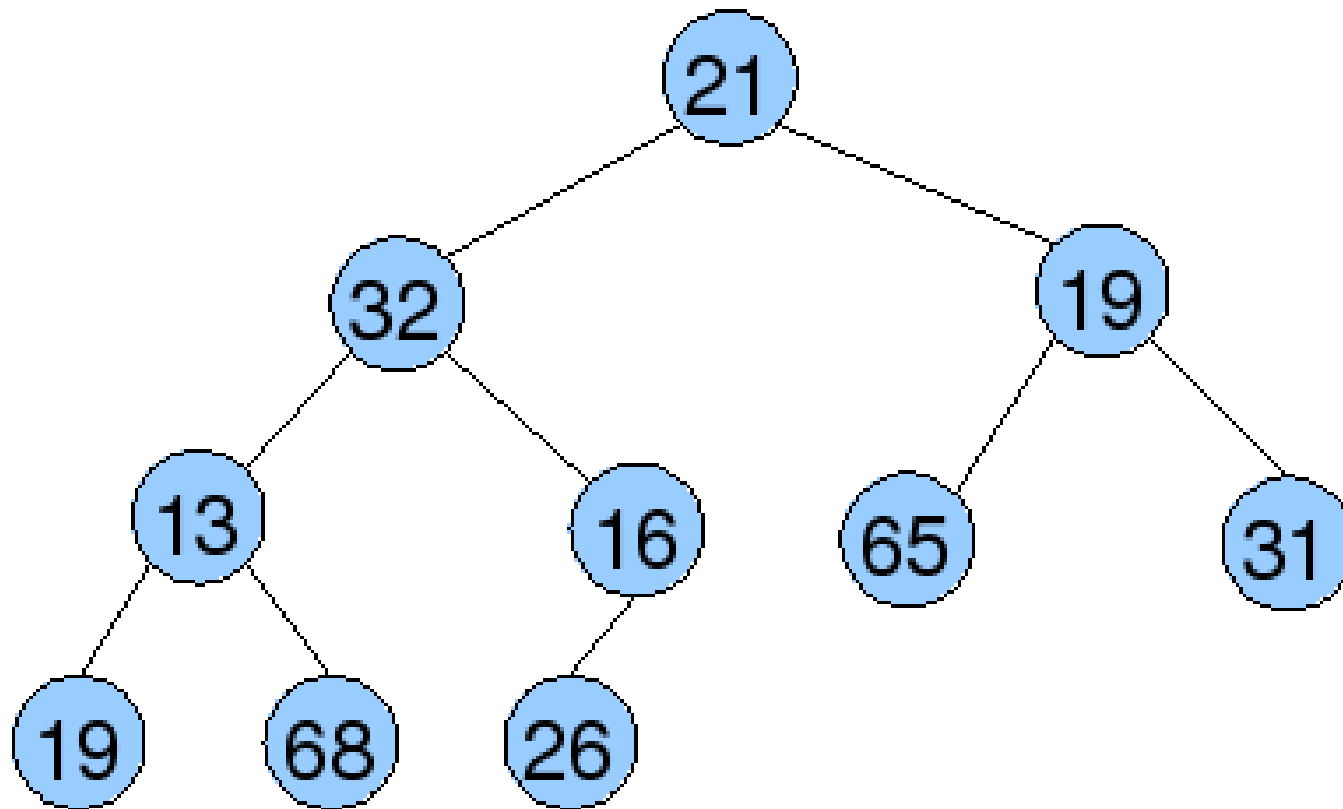
É Heap?



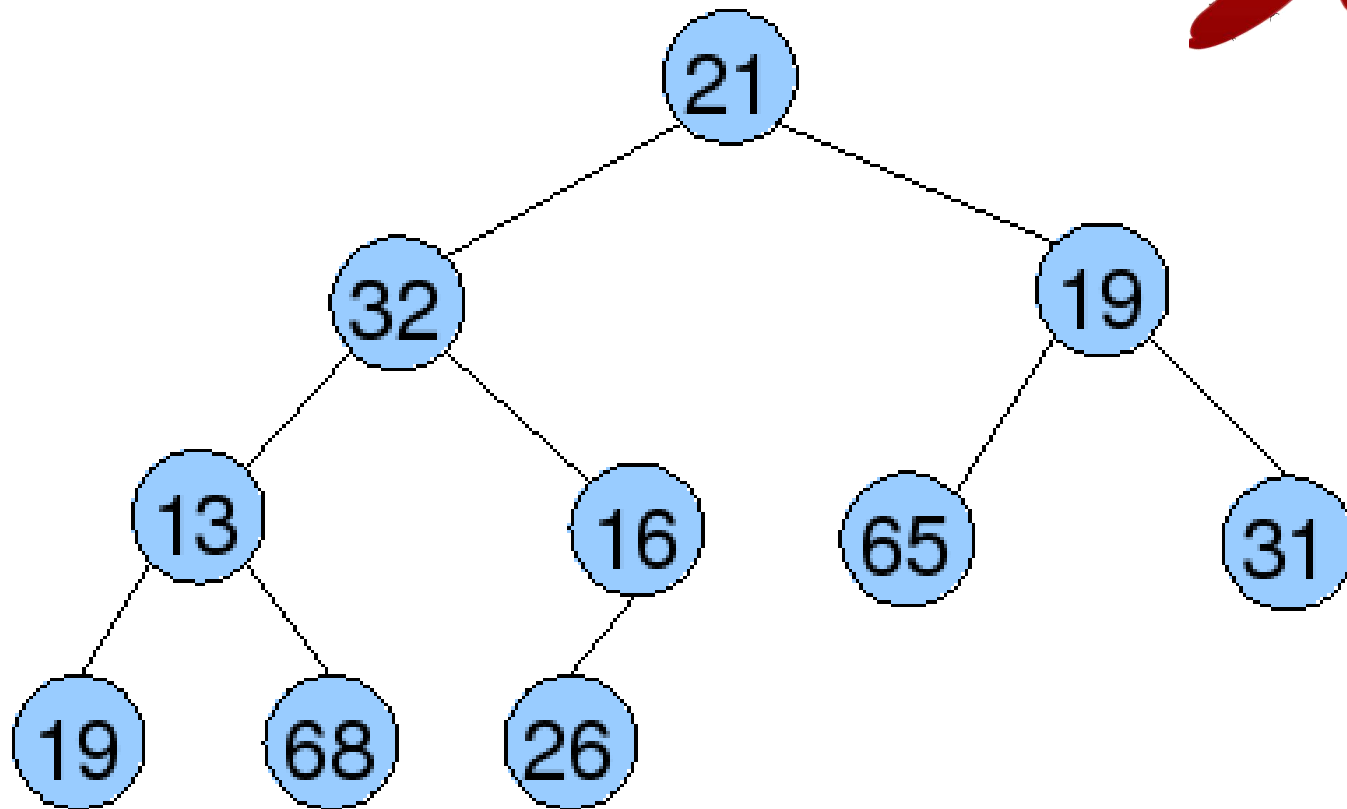
É Heap?



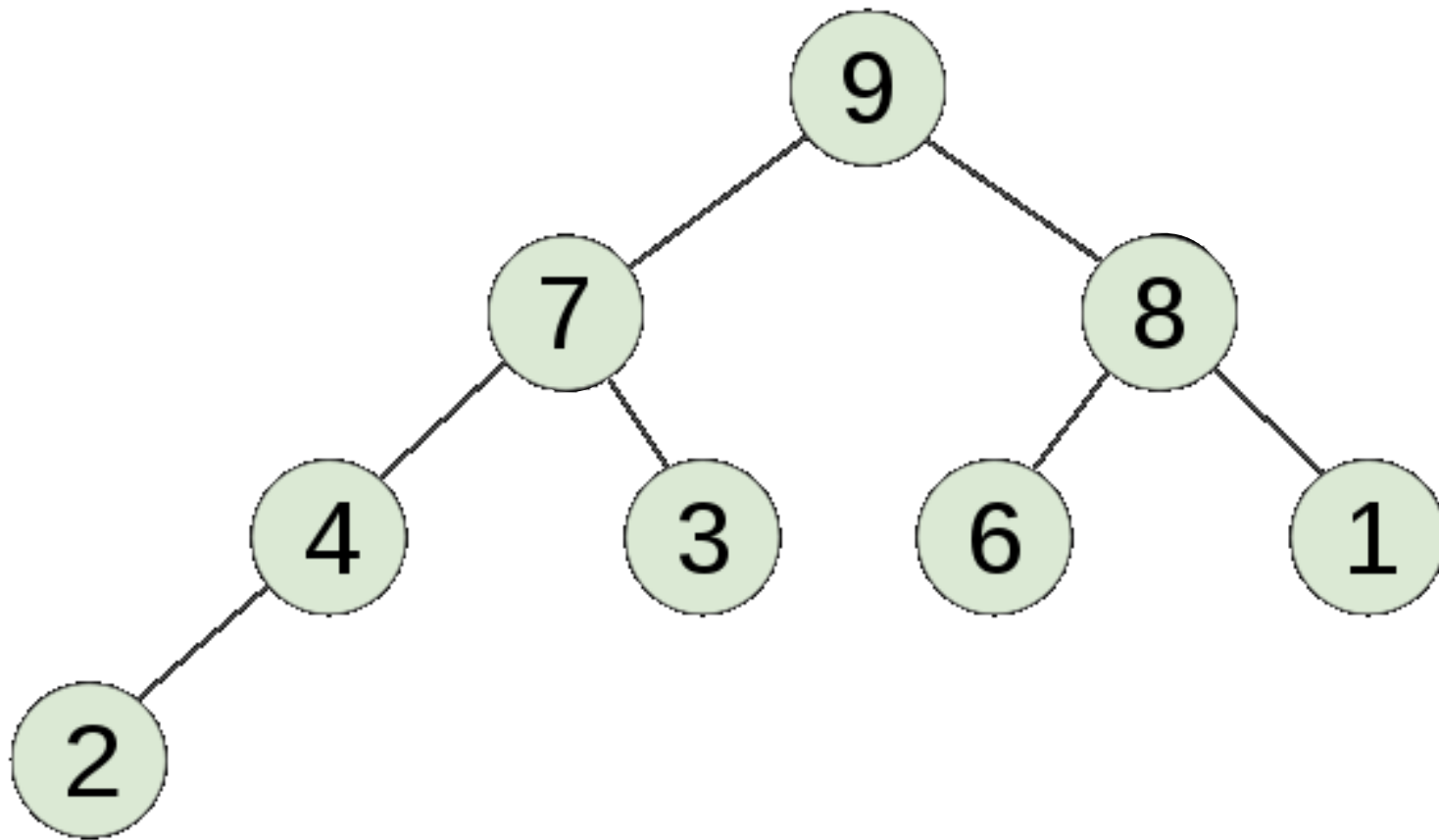
É Heap?



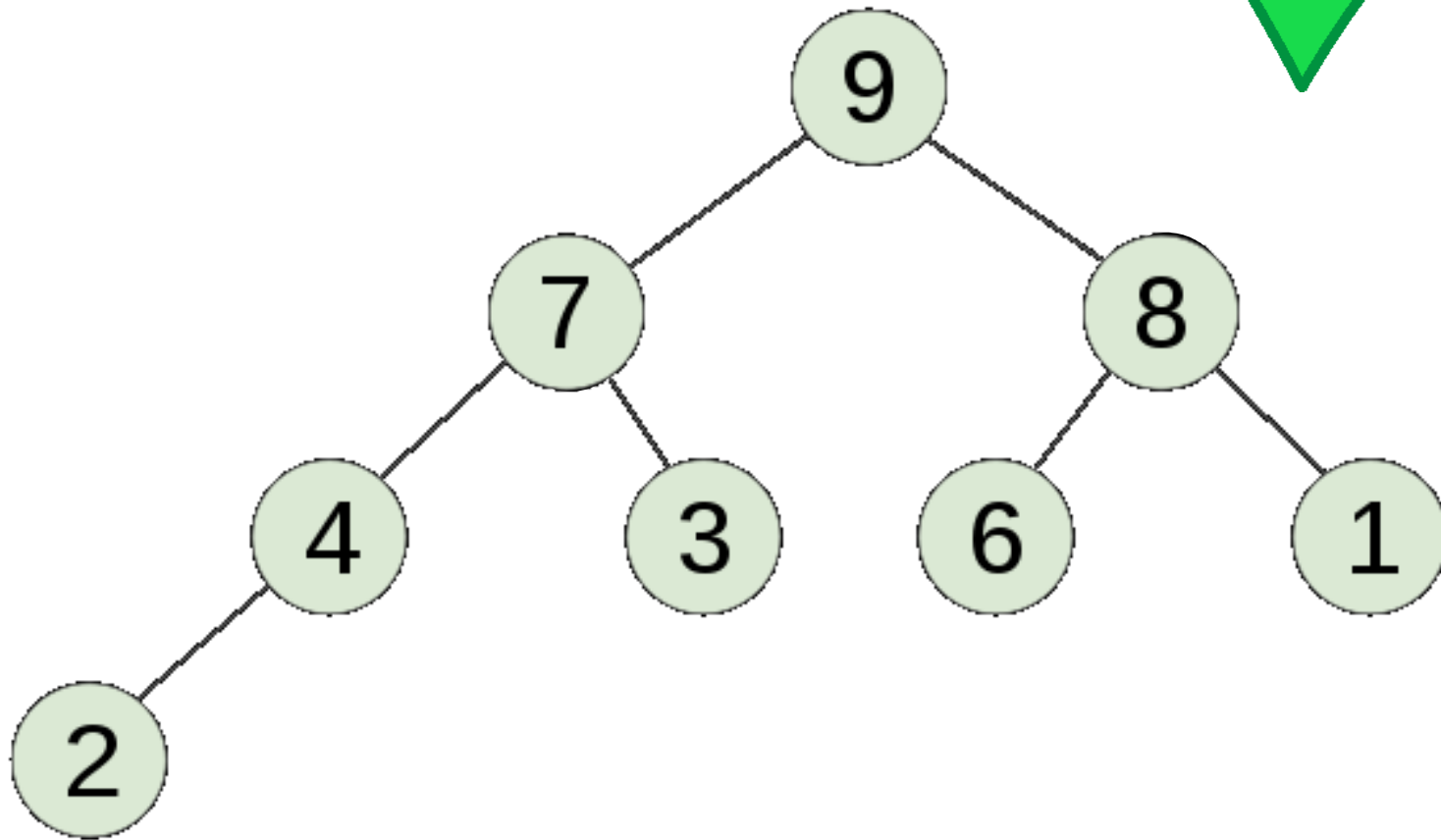
É Heap?



É Heap?



É Heap?



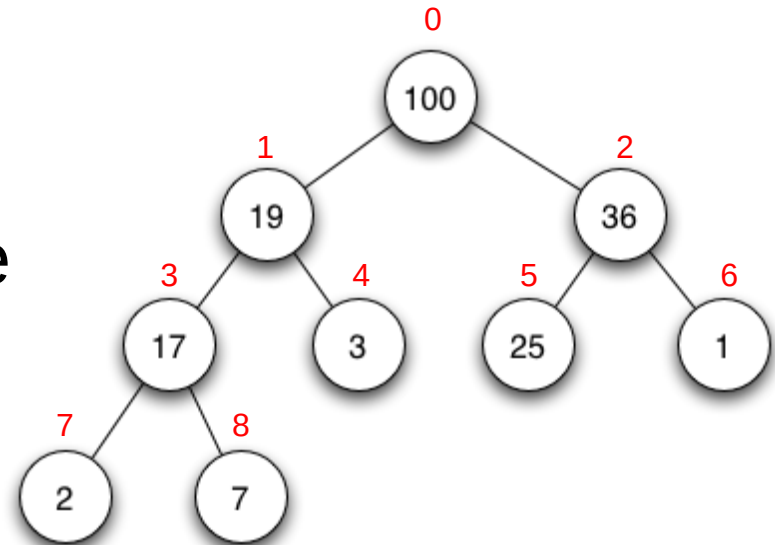
Heap Sort

- Dado o pai, encontrar os filhos:
 - Filho Esquerdo: $2 \cdot i + 1$
 - Filho Direito: $2 \cdot i + 2$
- Dado um filho, encontrar o pai:

$$\left\lfloor (i - 1) / 2 \right\rfloor$$

- A raiz encontra-se no índice **0** do vetor

0	1	2	3	4	5	6	7	8
100	19	36	17	3	25	1	2	7



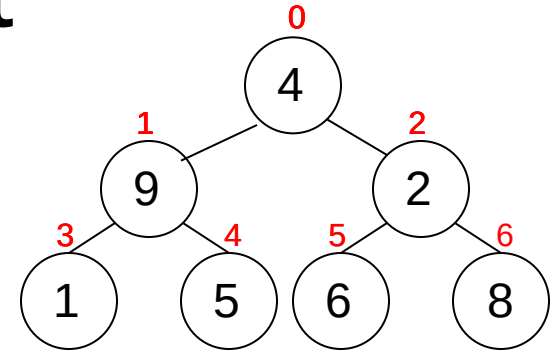
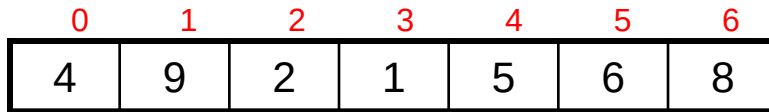
Heap Sort

Funcionamento:

1. Transformação de um vetor em um heap binário (constroi_Heap)
2. Ordenação
 1. A cada iteração obtem-se o maior elemento do heap (raiz do heap, indice 0 do vetor) e adicione-o em um segmento ordenado.
 2. Após a subtração da raiz, reorganiza-se o heap (restaura_heap)

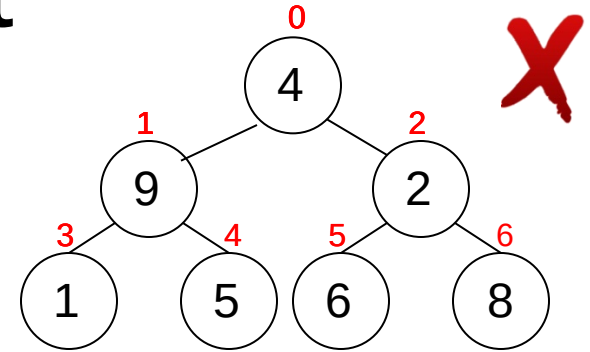
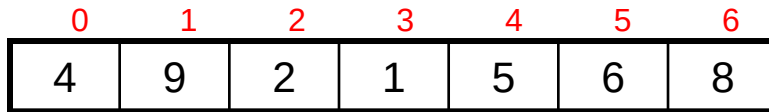
Heap Sort

Idéia:



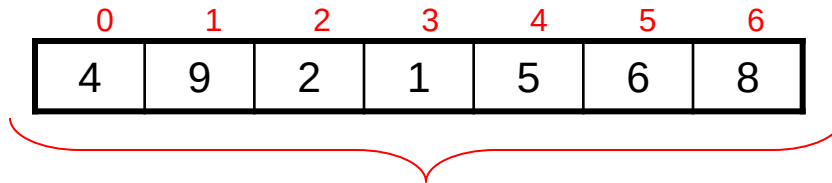
Heap Sort

Idéia:

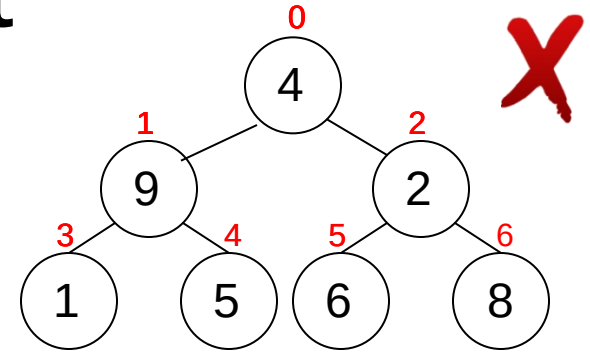


Heap Sort

Idéia:

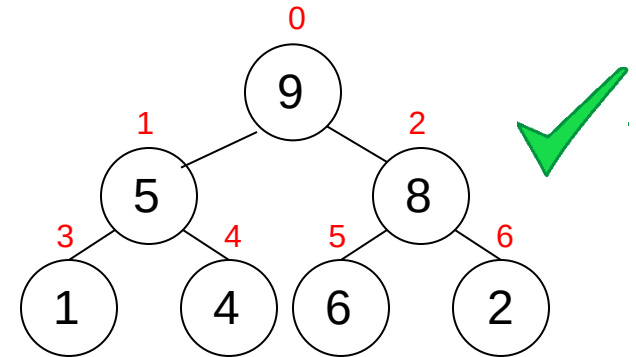
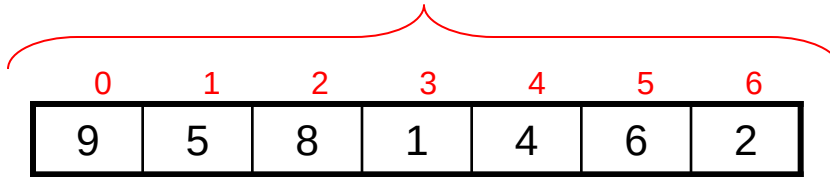
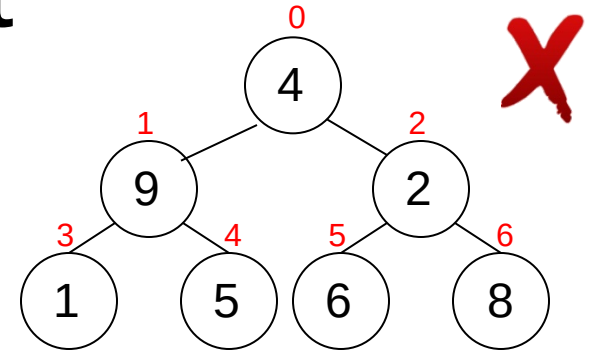
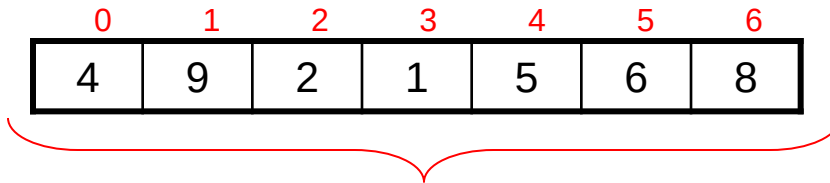


constroi_heap(...)



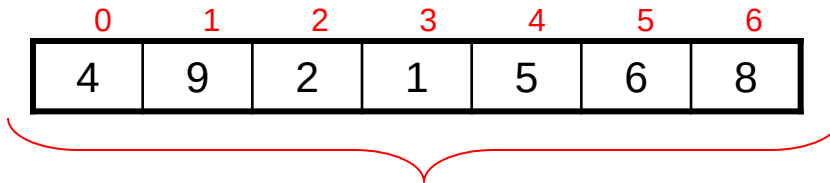
Heap Sort

Idéia:

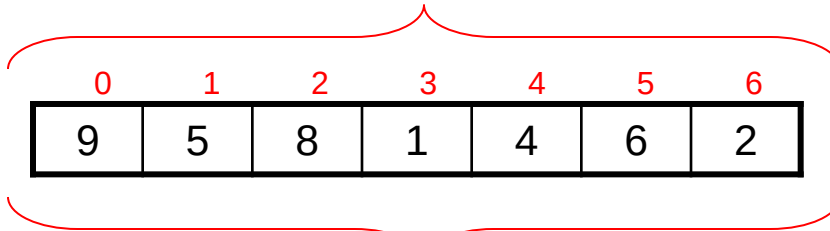


Heap Sort

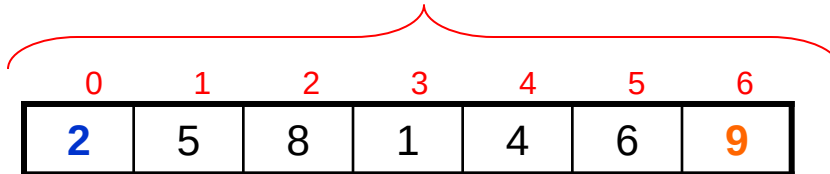
Idéia:



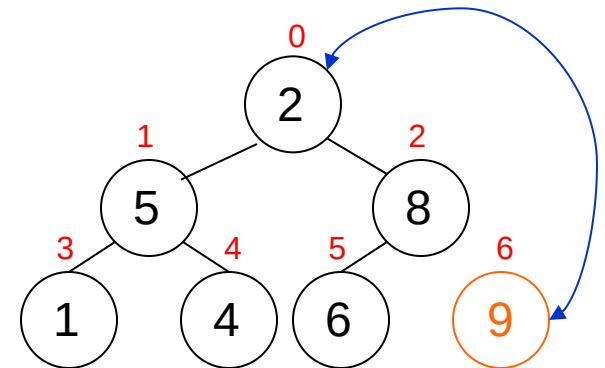
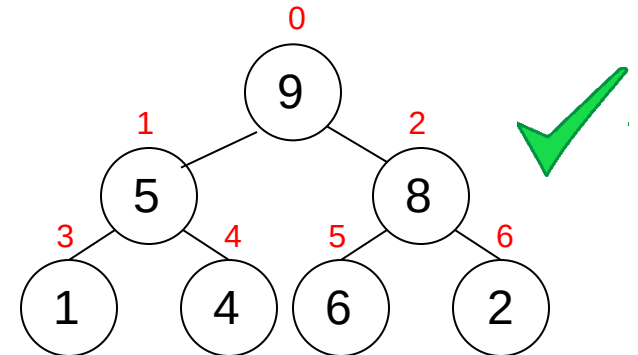
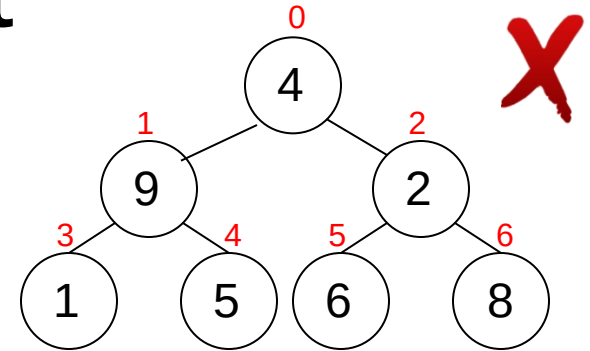
constroi_heap(...)



seleciona_max(...)

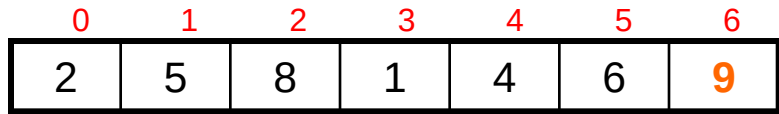


Seguimento ordenado

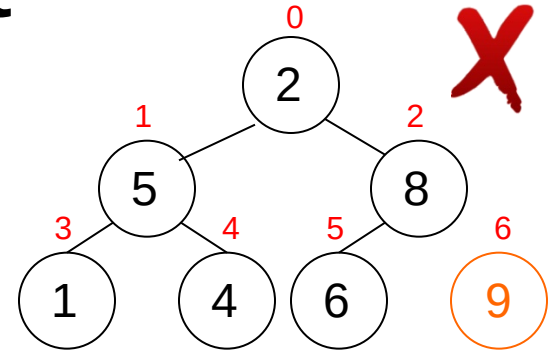


Heap Sort

Idéia:

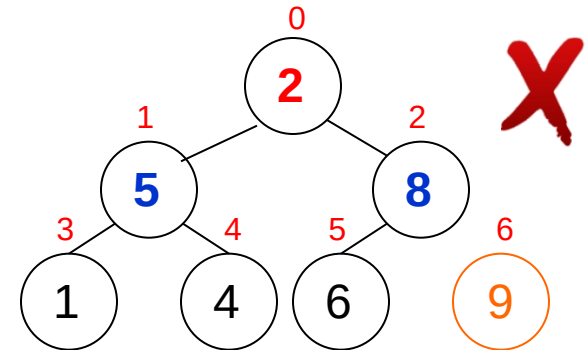
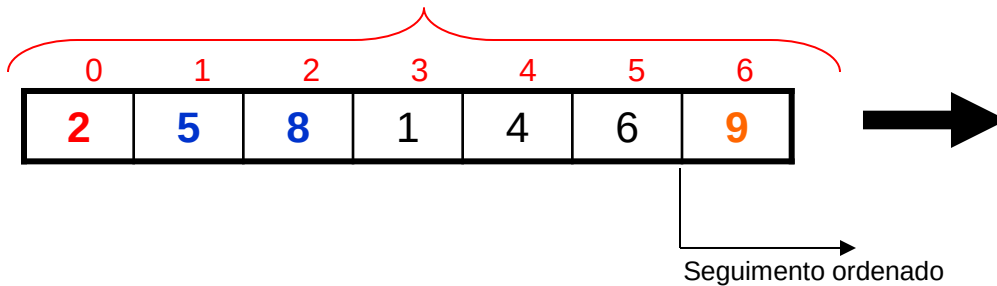
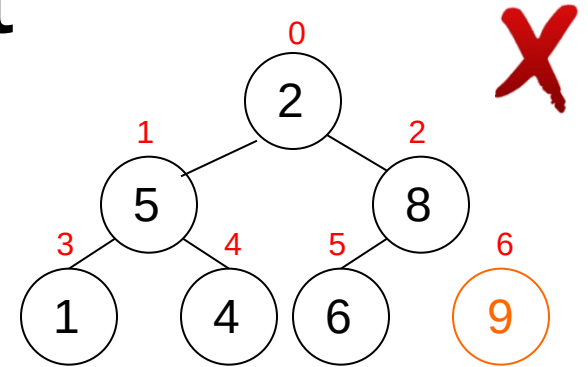
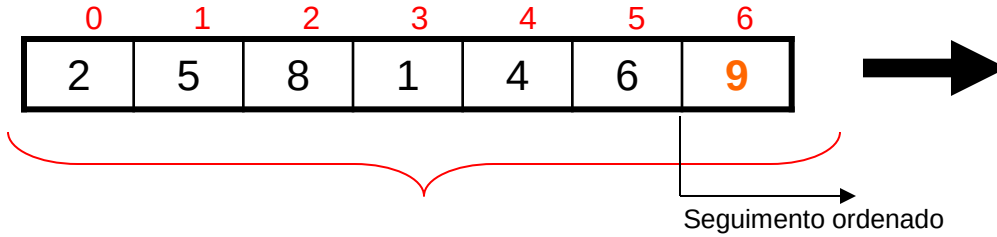


Seguimento ordenado



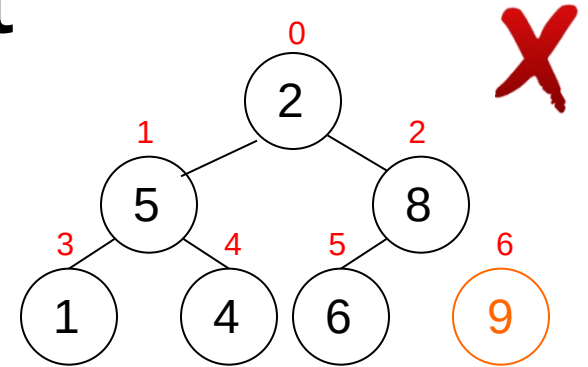
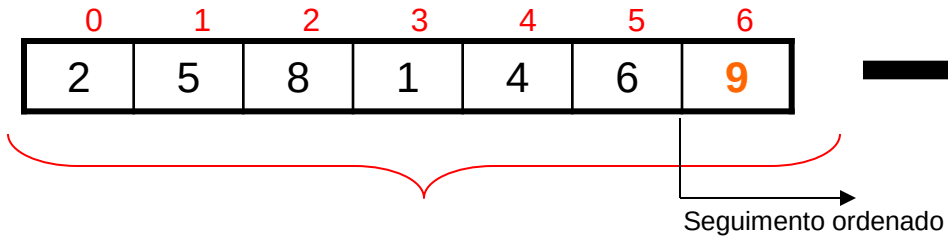
Heap Sort

Idéia:

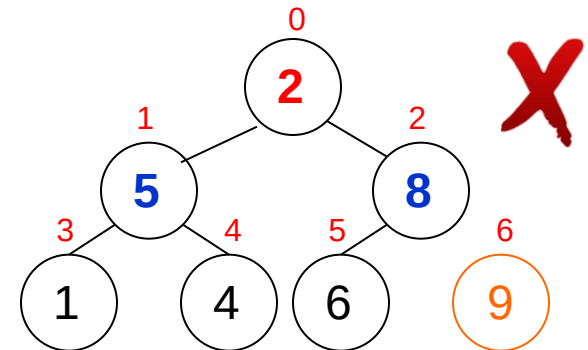
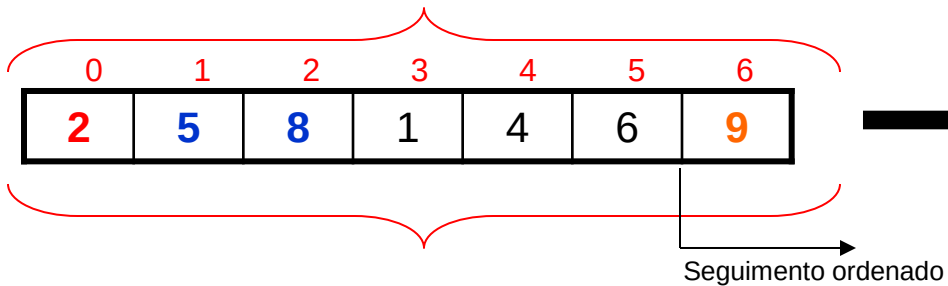


Heap Sort

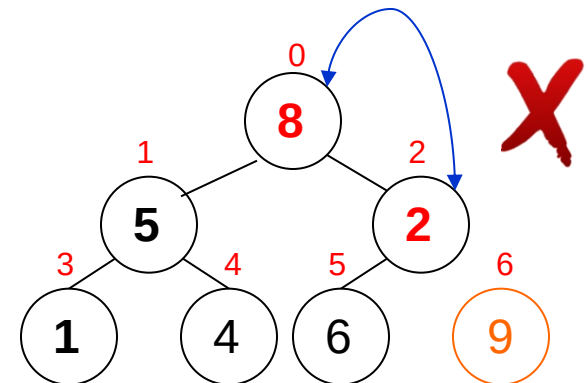
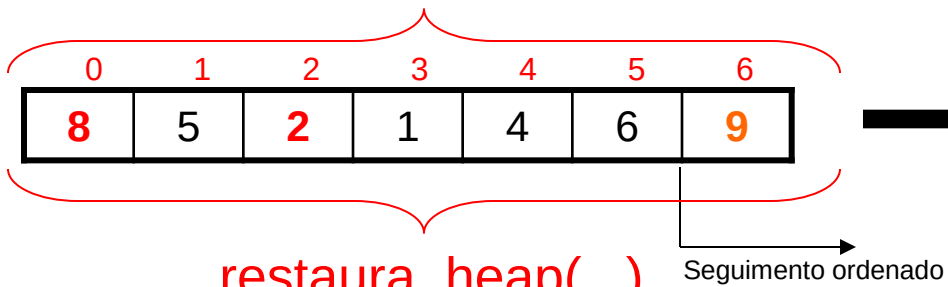
Idéia:



restaura_heap(...)



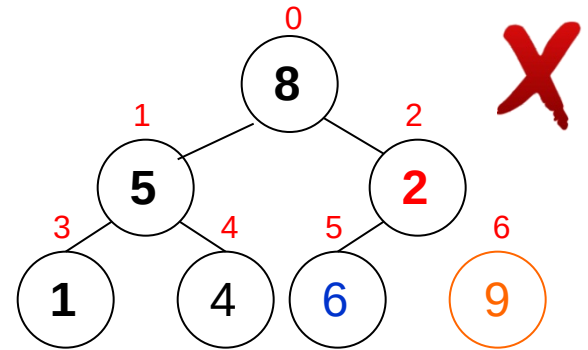
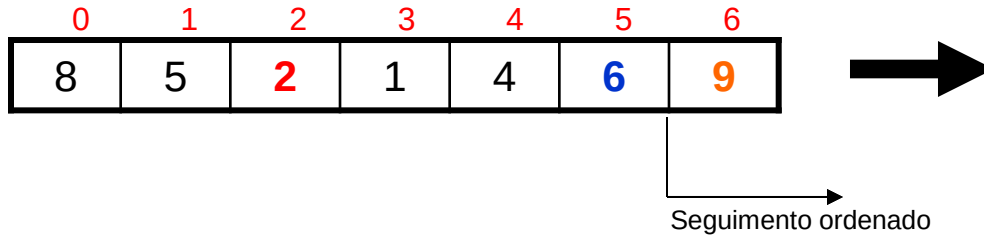
restaura_heap(...)



restaura_heap(...)

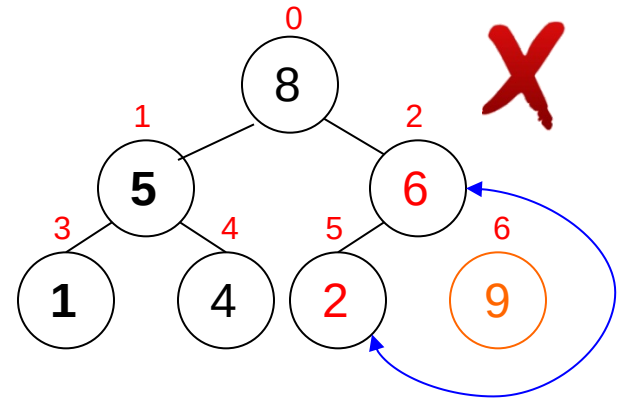
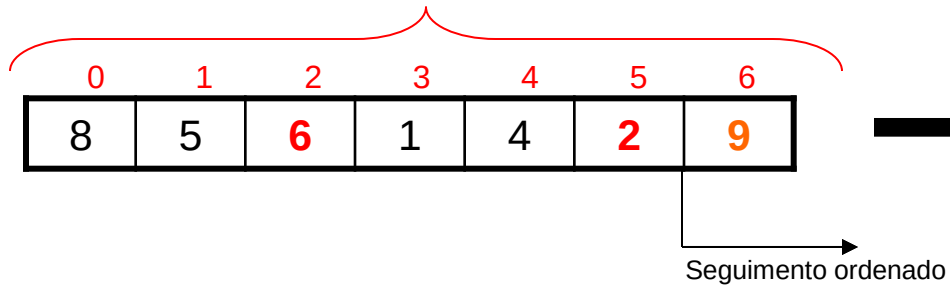
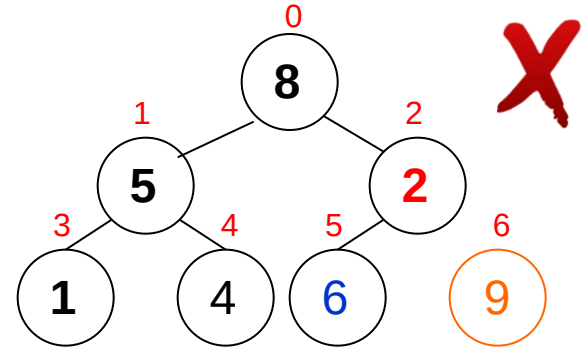
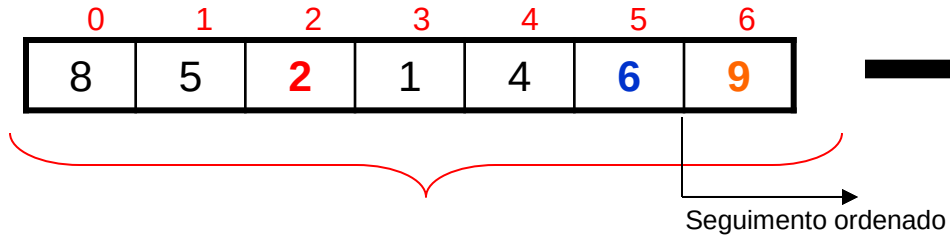
Heap Sort

Idéia:



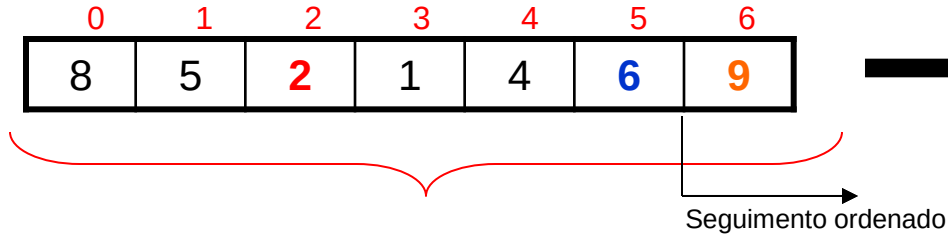
Heap Sort

Idéia:

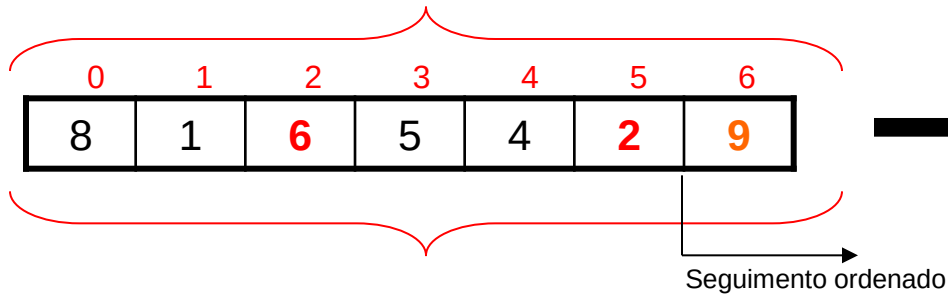


Heap Sort

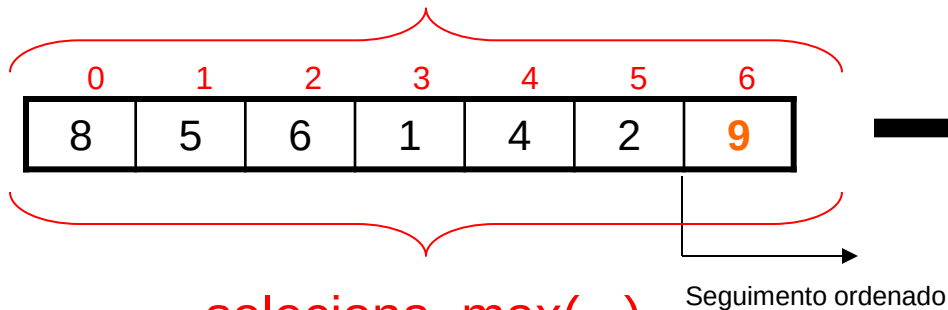
Idéia:



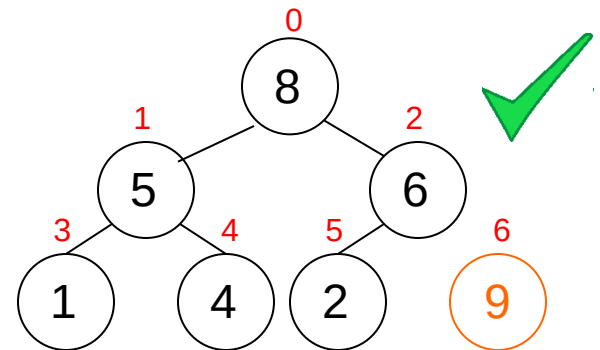
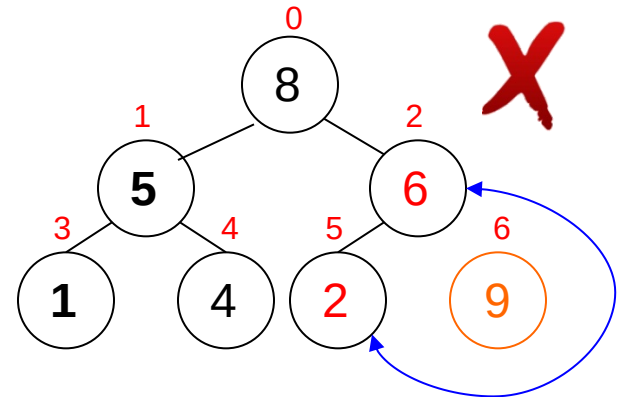
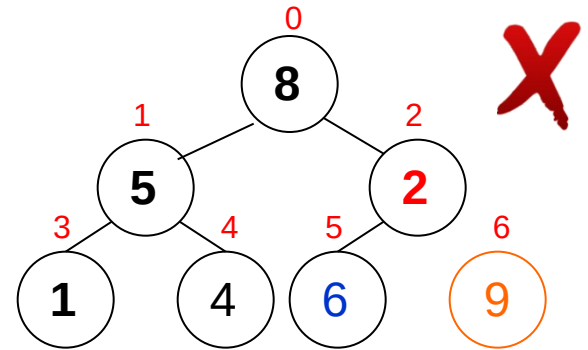
restaura_heap(...)



restaura_heap(...)



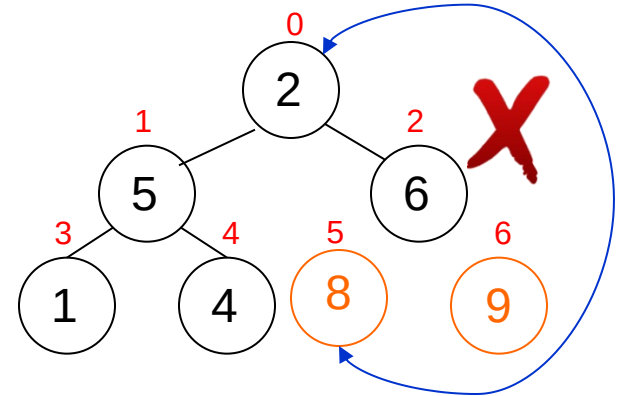
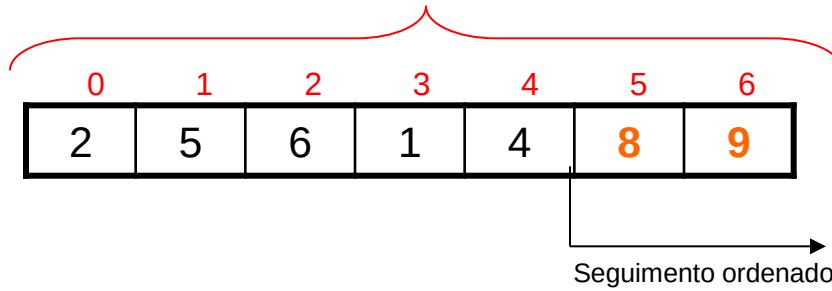
seleciona_max(...)



Heap Sort

Idéia:

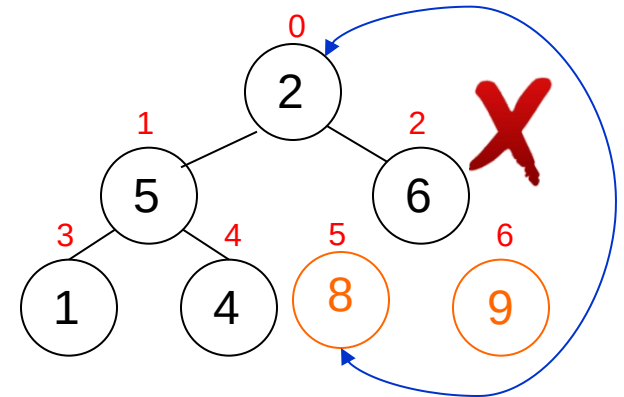
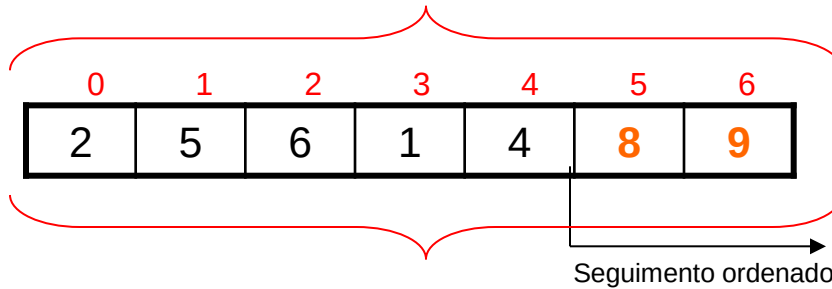
seleciona_max(...)



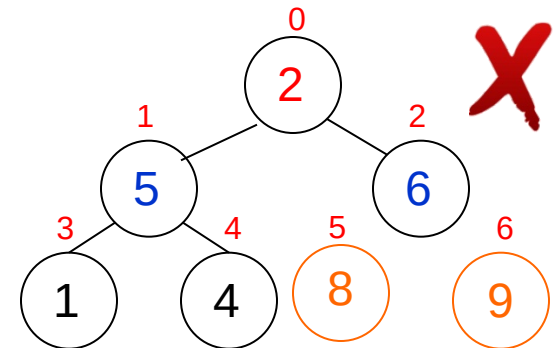
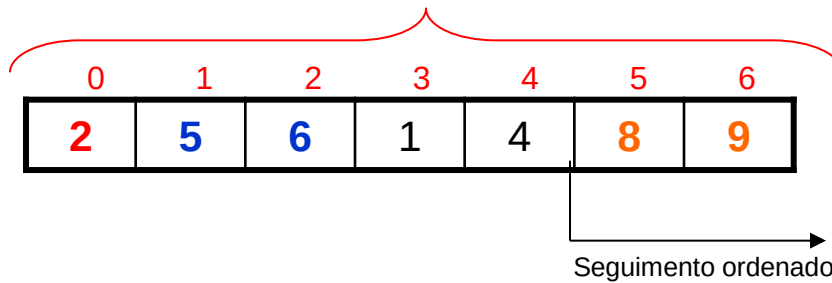
Heap Sort

Idéia:

seleciona_max(...)



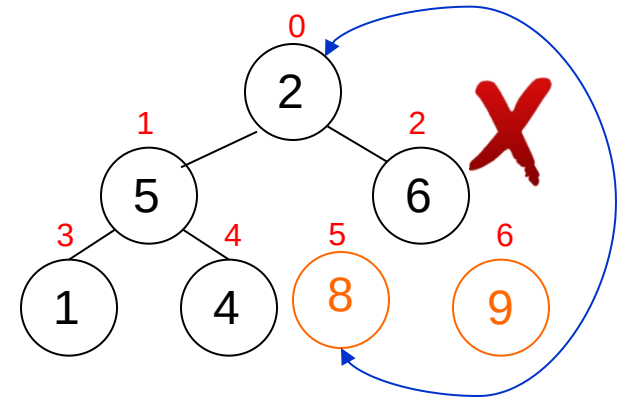
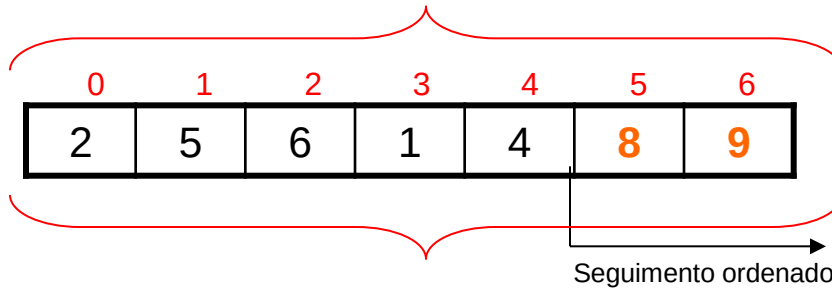
restaura_heap(...)



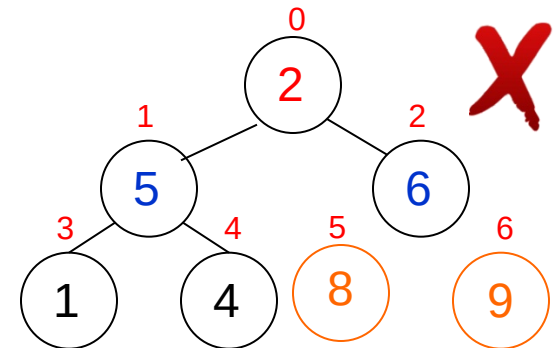
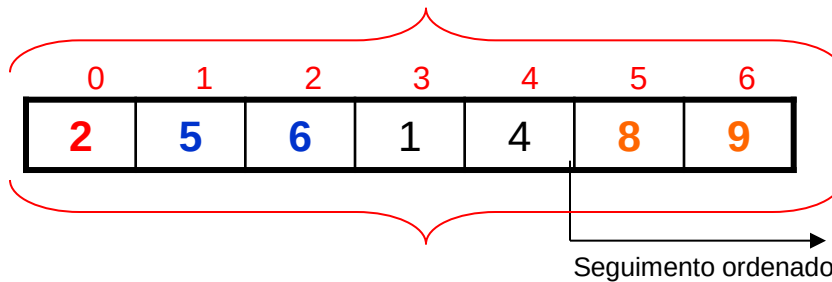
Heap Sort

Idéia:

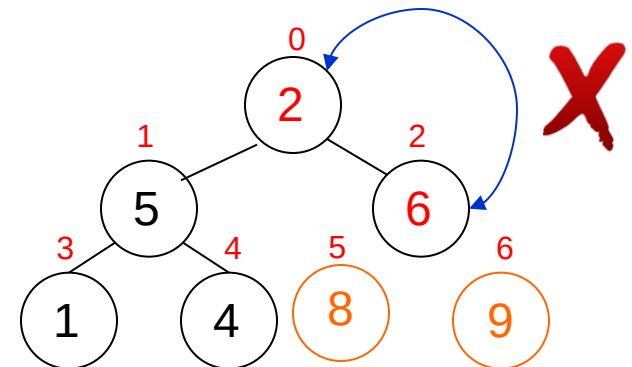
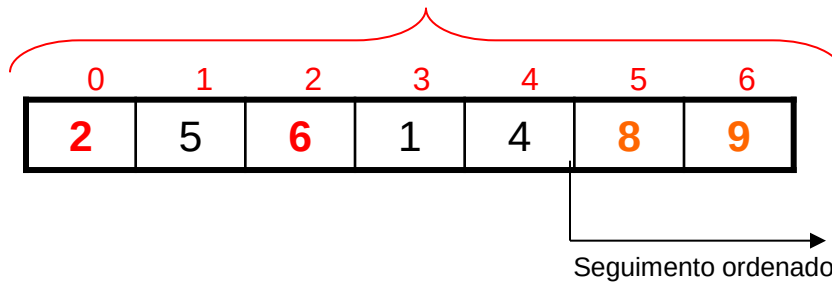
seleciona_max(...)



restaura_heap(...)

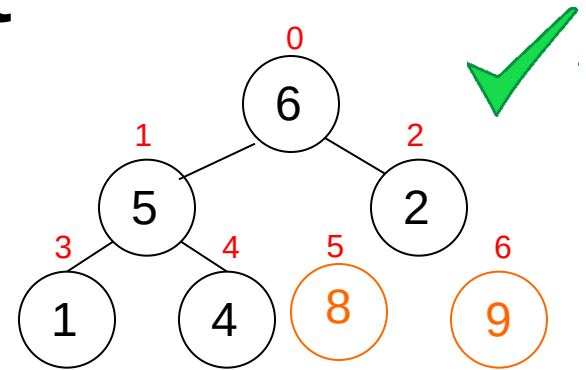
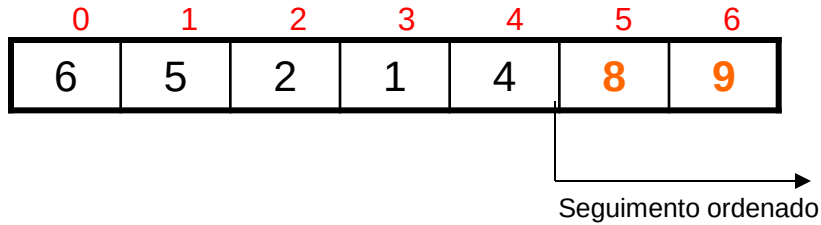


restaura_heap(...)



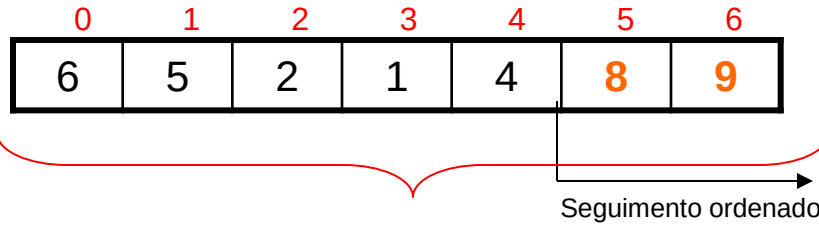
Heap Sort

Idéia:

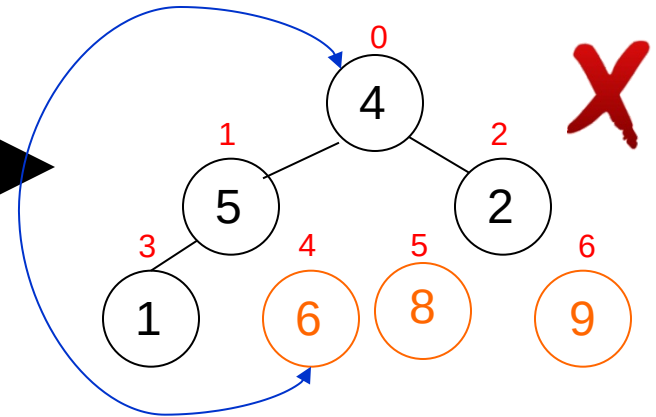
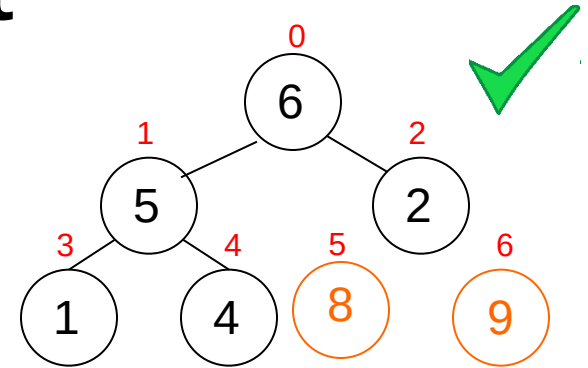
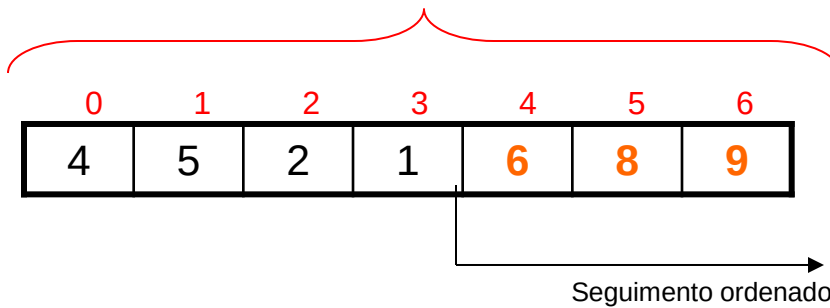


Heap Sort

Idéia:

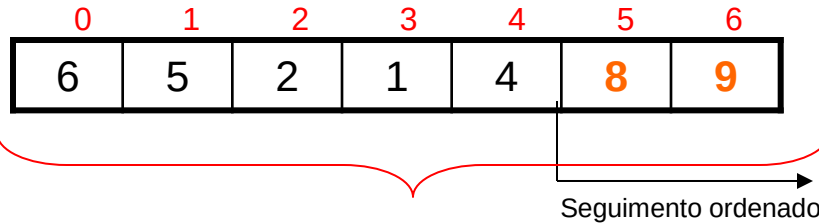


`seleciona_max(...)`

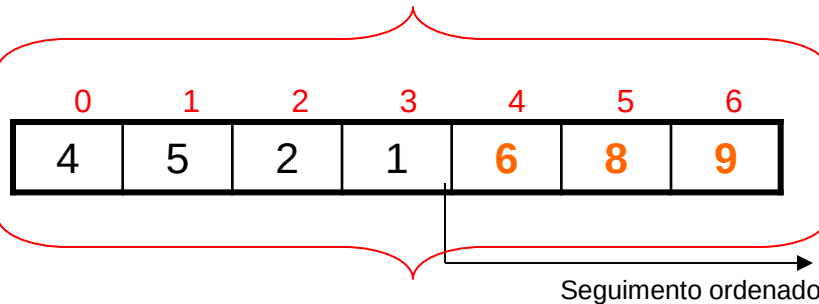
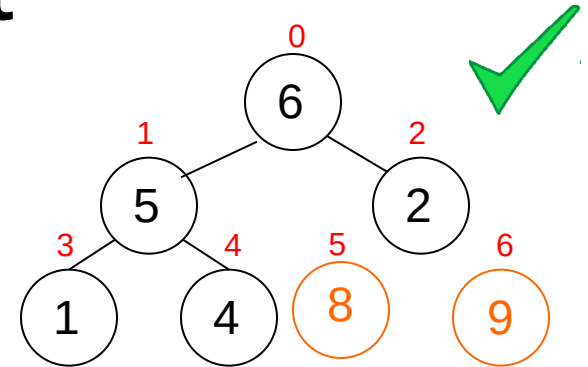


Heap Sort

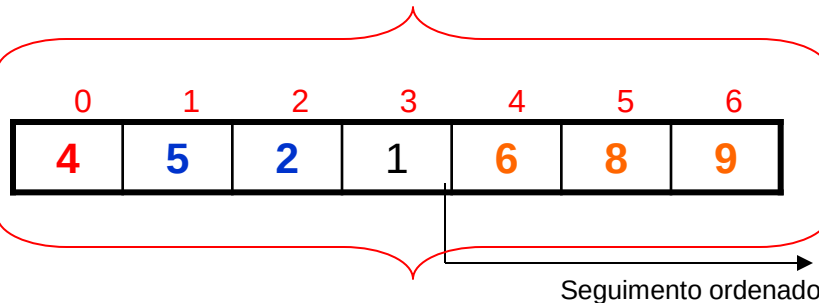
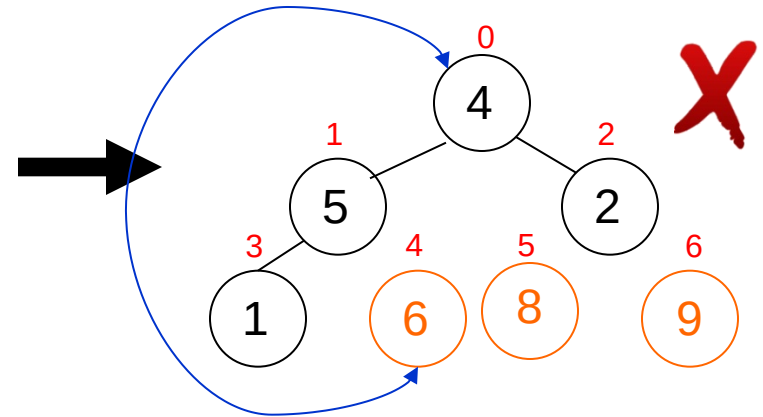
Idéia:



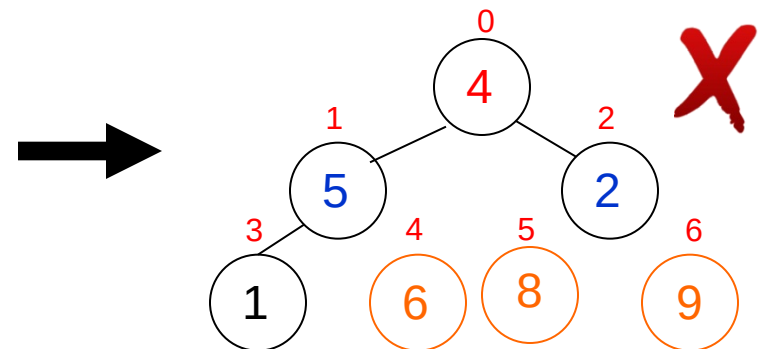
seleciona_max(...)



restaura_heap(...)

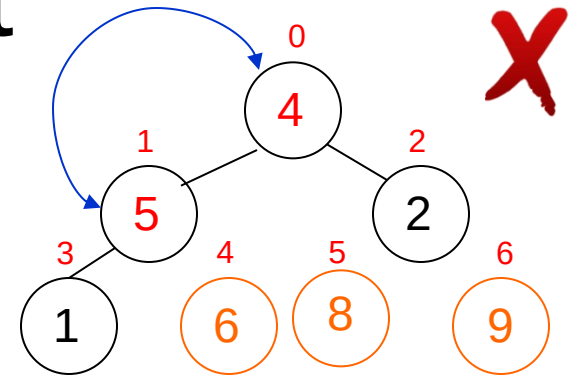
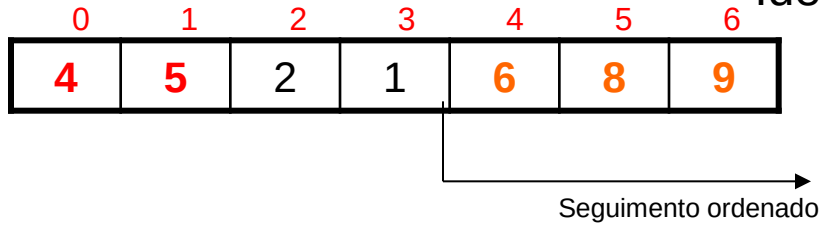


restaura_heap(...)



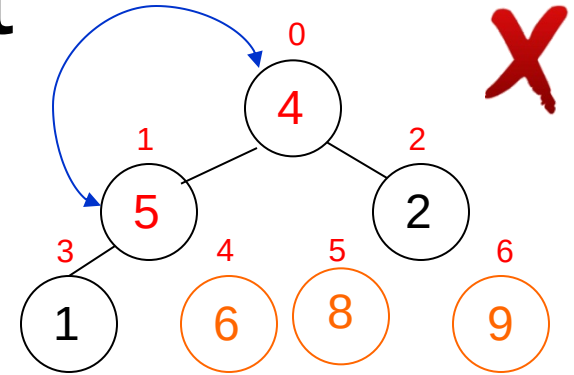
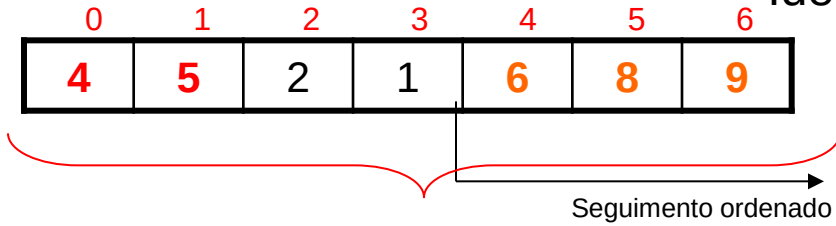
Heap Sort

Idéia:

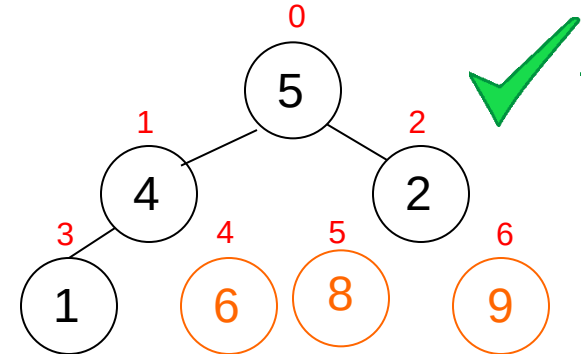
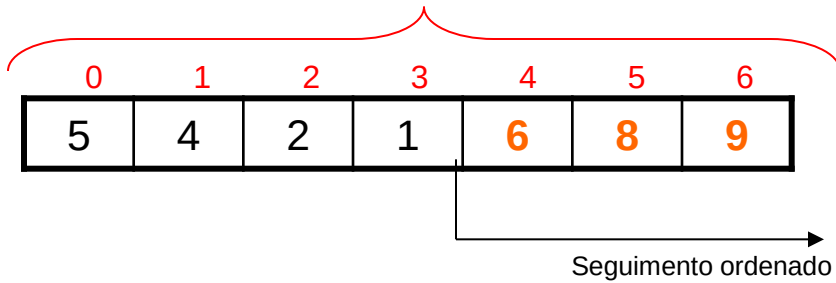


Heap Sort

Idéia:

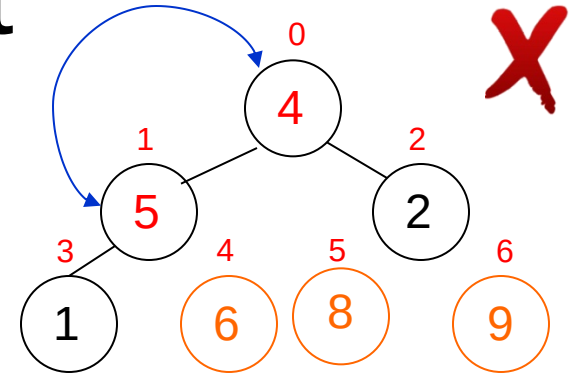
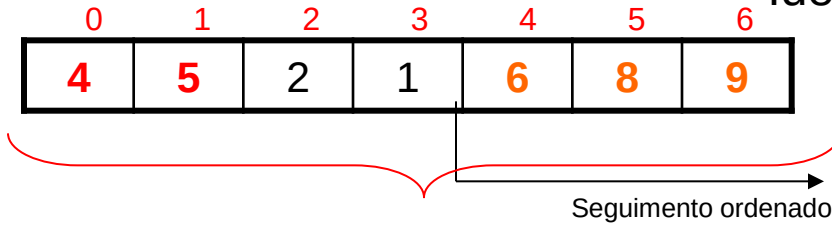


restaura_heap(...)

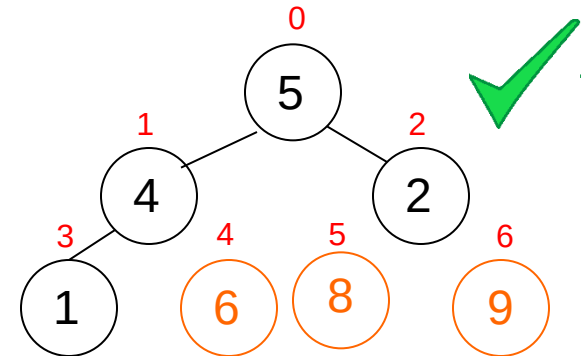
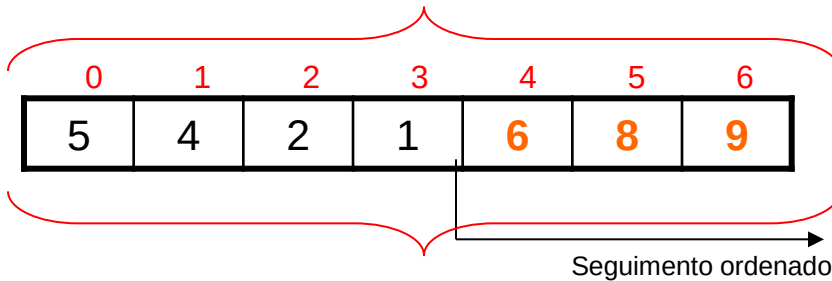


Heap Sort

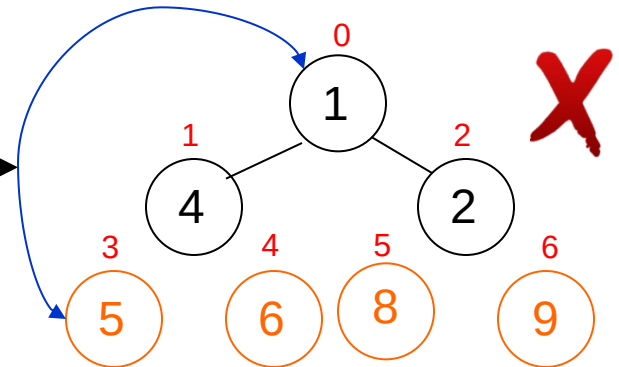
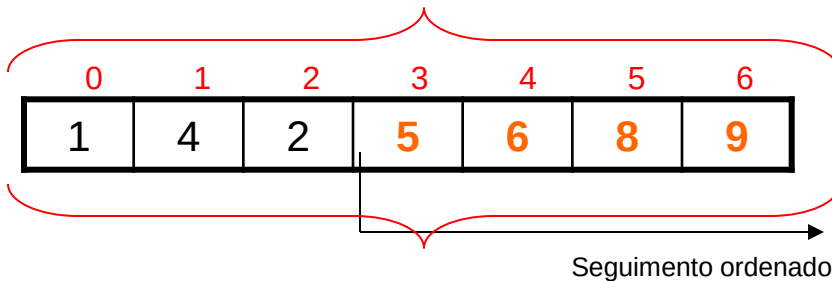
Idéia:



restaura_heap(...)



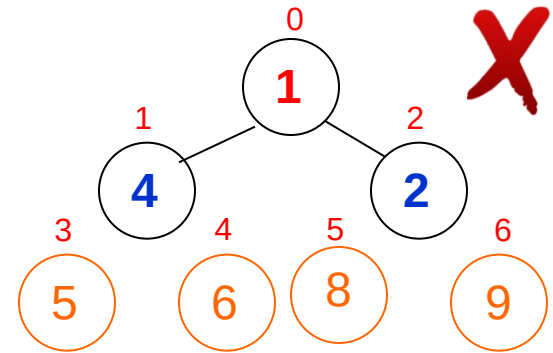
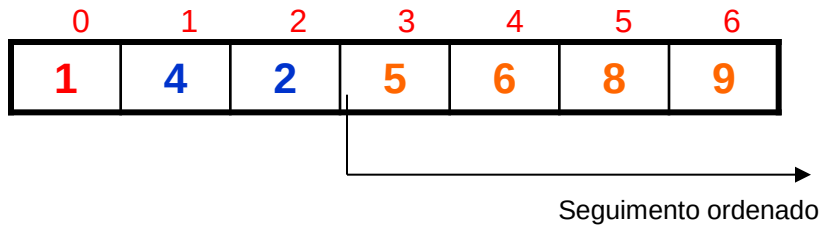
seleciona_max(...)



restaura_heap(...)

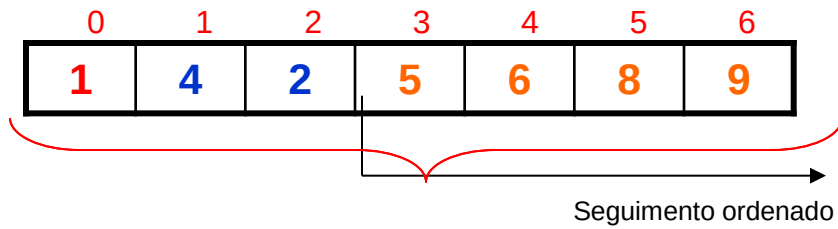
Heap Sort

Idéia:

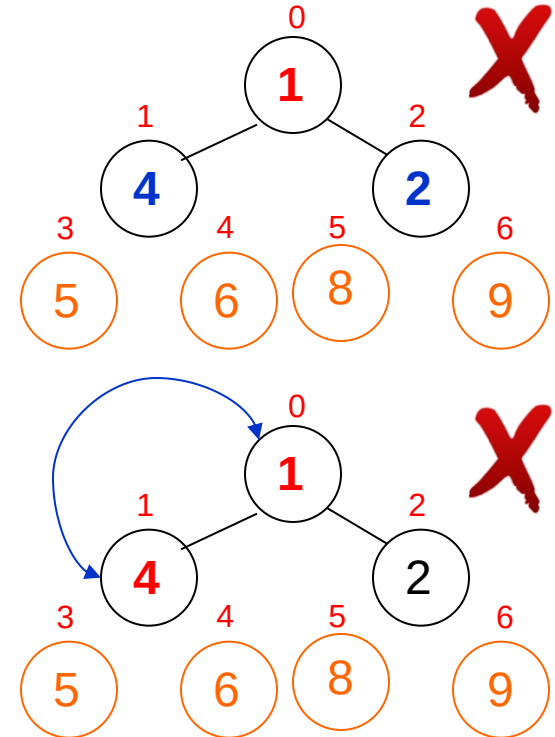
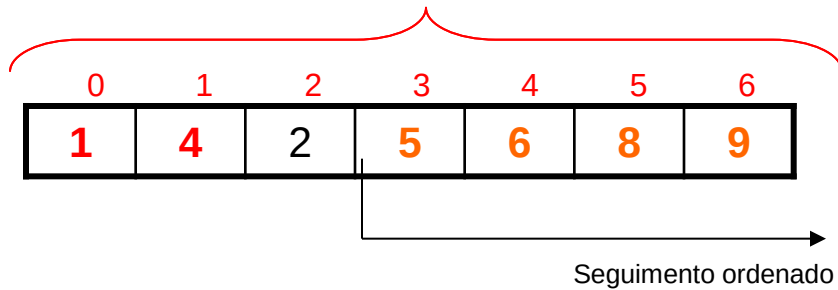


Heap Sort

Idéia:

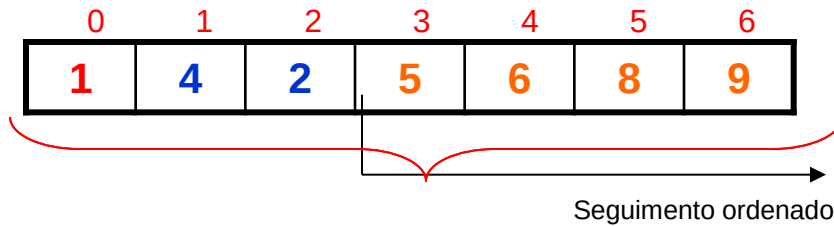


restaura_heap(...)

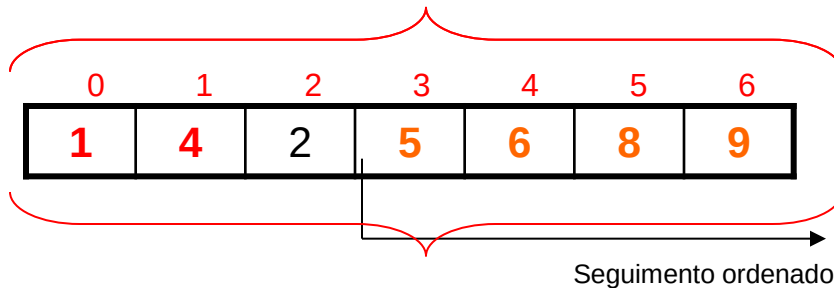


Heap Sort

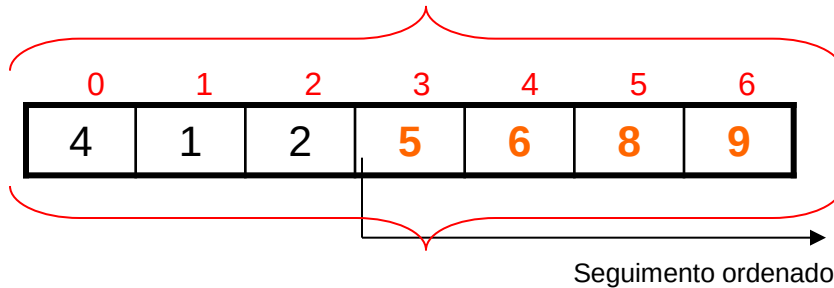
Idéia:



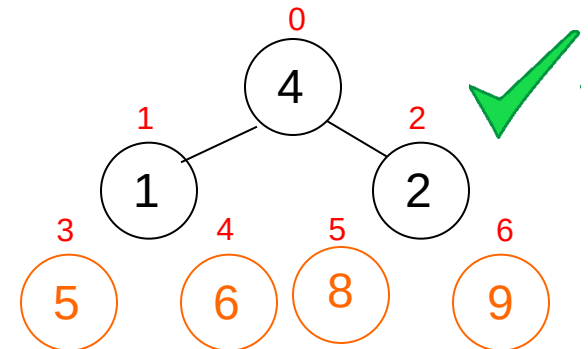
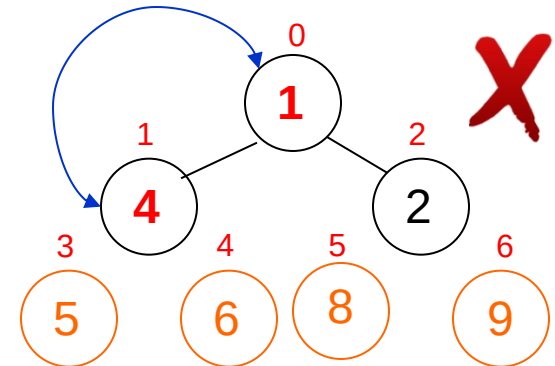
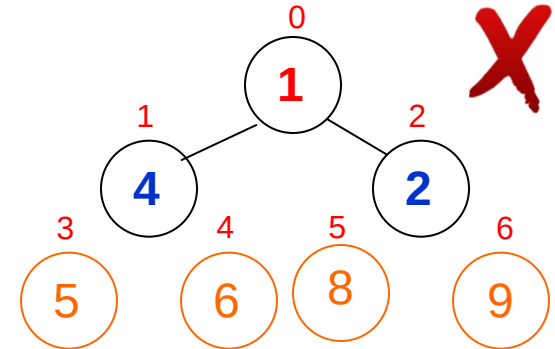
restaura_heap(...)



restaura_heap(...)

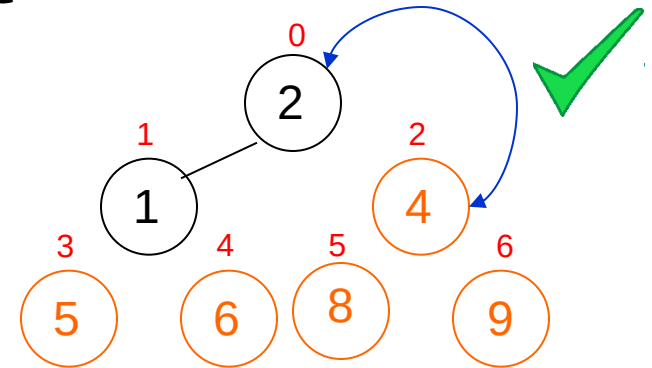
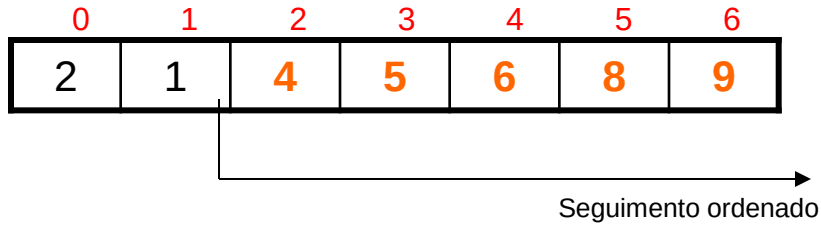


seleciona_max(...)



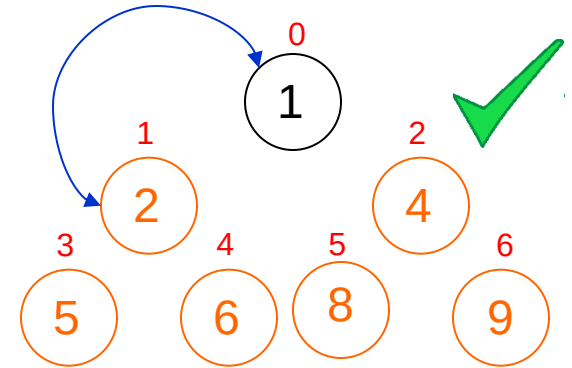
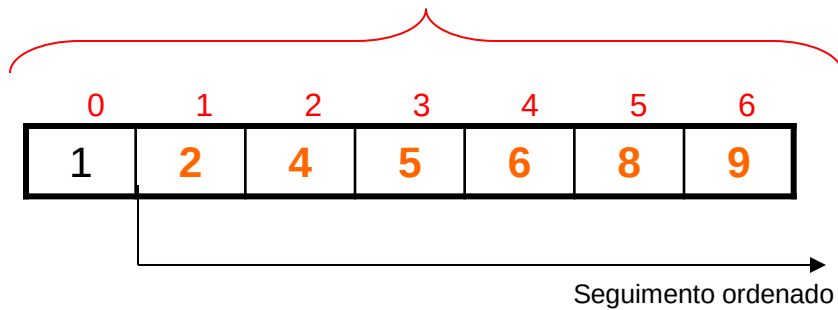
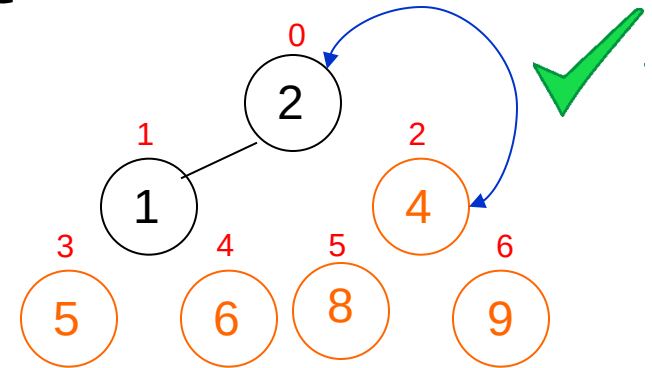
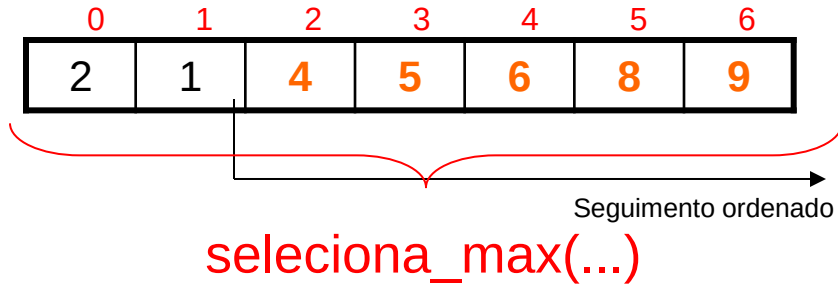
Heap Sort

Idéia:



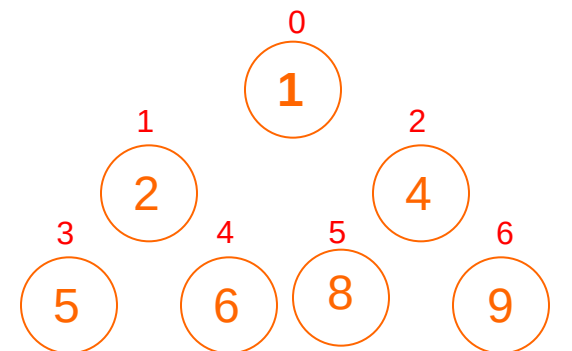
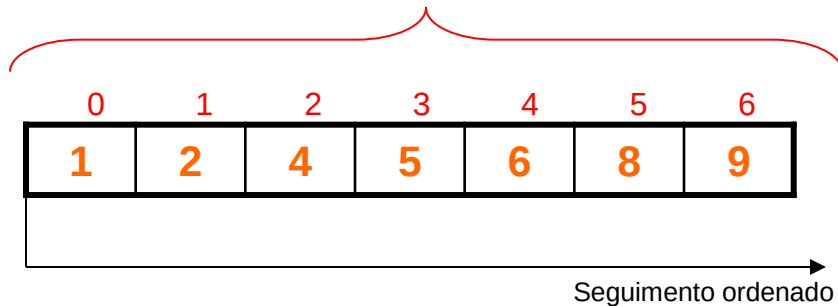
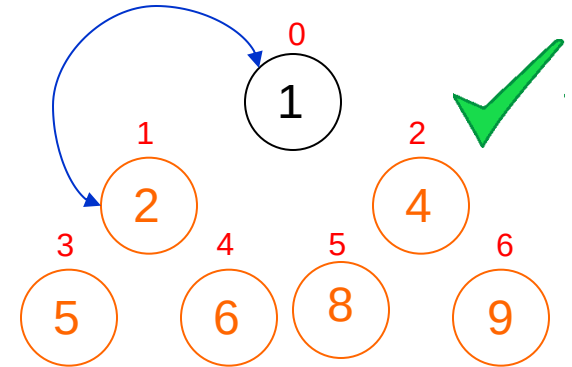
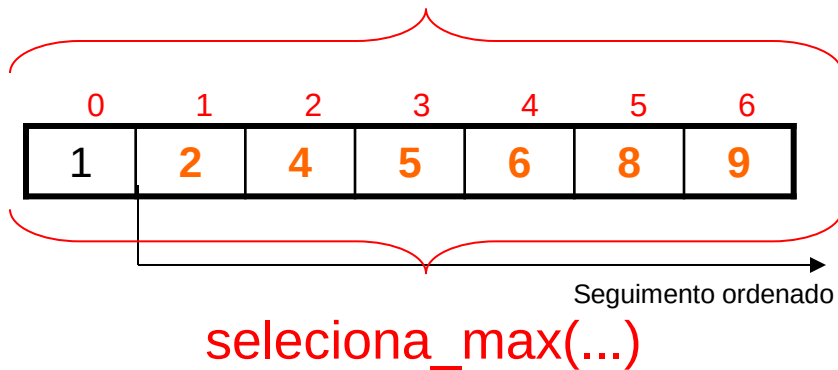
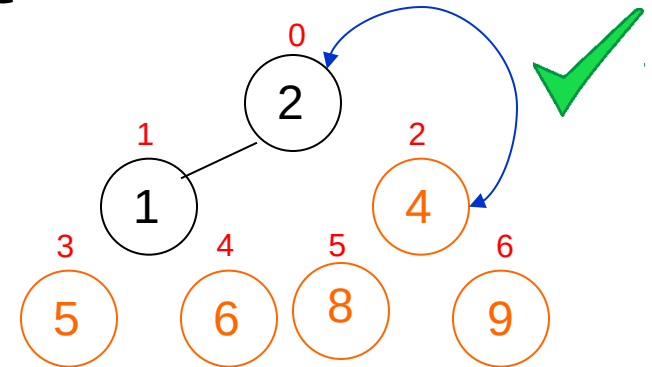
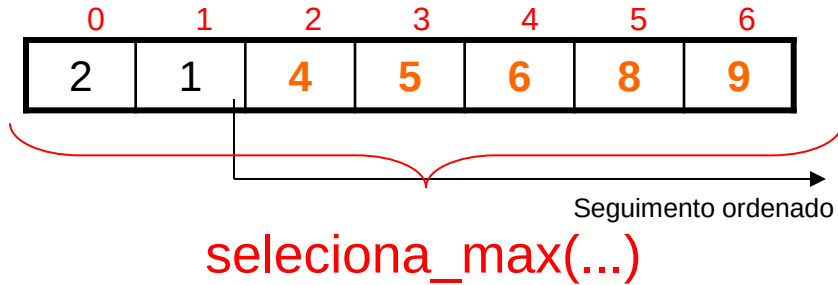
Heap Sort

Idéia:



Heap Sort

Idéia:



Heap Sort

Algoritmo:

FilhoEsquerdo(iPai)

1. $fEsq \leftarrow 2 * iPai + 1$

2. **retorne** fEsq

Heap Sort

Algoritmo:

FilhoEsquerdo(iPai)

1. $fEsq \leftarrow 2 * iPai + 1$
2. **retorne** fEsq

FilhoDireito(iPai)

1. $fDir \leftarrow 2 * iPai + 2$
2. **retorne** fDir

Heap Sort

Algoritmo:

FilhoEsquerdo(iPai)

1. $fEsq \leftarrow 2 * iPai + 1$
2. **retorne** fEsq

FilhoDireito(iPai)

1. $fDir \leftarrow 2 * iPai + 2$
2. **retorne** fDir

SelecionaMaximo(vetor[0, ..., n], tamanho)

1. $aux \leftarrow vetor[0]$
2. $vetor[0] \leftarrow vetor[tamanho]$
3. $vetor[tamanho] \leftarrow aux$
4. $tamanho \leftarrow tamanho - 1$
5. **retorne** tamanho

Heap Sort

Algoritmo:

FilhoEsquerdo(iPai)

1. $fEsq \leftarrow 2 * iPai + 1$
2. **retorne** fEsq

FilhoDireito(iPai)

1. $fDir \leftarrow 2 * iPai + 2$
2. **retorne** fDir

SelecionaMaximo(vetor[0, ..., n], tamanho)

1. $aux \leftarrow vetor[0]$
2. $vetor[0] \leftarrow vetor[tamanho]$
3. $vetor[tamanho] \leftarrow aux$
4. $tamanho \leftarrow tamanho - 1$
5. **retorne** tamanho

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$
2. $fEsq \leftarrow FilhoEsquerdo(ind)$
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então**
4. $maior \leftarrow fEsq$
5. **senão**
6. $maior \leftarrow ind$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então**
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow vetor[ind]$
13. $v[ind] \leftarrow v[maior]$
14. $v[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo:

FilhoEsquerdo(iPai)

1. $fEsq \leftarrow 2 * iPai + 1$
2. **retorne** fEsq

FilhoDireito(iPai)

1. $fDir \leftarrow 2 * iPai + 2$
2. **retorne** fDir

SelecionaMaximo(vetor[0, ..., n], tamanho)

1. $aux \leftarrow vetor[0]$
2. $vetor[0] \leftarrow vetor[tamanho]$
3. $vetor[tamanho] \leftarrow aux$
4. $tamanho \leftarrow tamanho - 1$
5. **retorne** tamanho

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$
2. $fEsq \leftarrow FilhoEsquerdo(ind)$
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então**
4. $maior \leftarrow fEsq$
5. **senão**
6. $maior \leftarrow ind$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então**
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow vetor[ind]$
13. $v[ind] \leftarrow v[maior]$
14. $v[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

ConstroiHeap(vetor[0, ..., n], tamanho)

1. **para** i **de** $div(tamanho, 2) - 1$ **até** 0 **passo** -1 **faça**
2. RestauraHeap(vetor, tamanho, i)
3. **fim-para**

Heap Sort

Algoritmo:

FilhoEsquerdo(iPai)

1. $fEsq \leftarrow 2 * iPai + 1$
2. **retorne** fEsq

FilhoDireito(iPai)

1. $fDir \leftarrow 2 * iPai + 2$
2. **retorne** fDir

SelecionaMaximo(vetor[0, ..., n], tamanho)

1. $aux \leftarrow vetor[0]$
2. $vetor[0] \leftarrow vetor[tamanho]$
3. $vetor[tamanho] \leftarrow aux$
4. $tamanho \leftarrow tamanho - 1$
5. **retorne** tamanho

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho)
2. **para** i **de** tamanho-1 **até** 0 **passo** -1 **faça**
3. $tamanho \leftarrow SelecionaMaximo(vetor, tamanho)$
4. RestauraHeap(vetor, tamanho, 0)
5. **fim-para**

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$
2. $fEsq \leftarrow FilhoEsquerdo(ind)$
3. **se** fEsq < tamanho **e** vetor[fEsq] > vetor[ind] **então**
4. $maior \leftarrow fEsq$
5. **senão**
6. $maior \leftarrow ind$
7. **fim-se**
8. **se** fDir < tamanho **e** vetor[fDir] > vetor[maior] **então**
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** maior $\neq$ ind **então**
12. $aux \leftarrow vetor[ind]$
13. $v[ind] \leftarrow v[maior]$
14. $v[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

ConstroiHeap(vetor[0, ..., n], tamanho)

1. **para** i **de** $div(tamanho, 2) - 1$ **até** 0 **passo** -1 **faça**
2. RestauraHeap(vetor, tamanho, i)
3. **fim-para**

Heap Sort

Algoritmo:

FilhoEsquerdo(iPai)

1. $fEsq \leftarrow 2 * iPai + 1$
2. **retorne** fEsq

Retorna a posição do Filho Esquerdo do nó iPai.

FilhoDireito(iPai)

1. $fDir \leftarrow 2 * iPai + 2$
2. **retorne** fDir

SelecionaMaximo(vetor[0, ..., n], tamanho)

1. $aux \leftarrow vetor[0]$
2. $vetor[0] \leftarrow vetor[tamanho]$
3. $vetor[tamanho] \leftarrow aux$
4. $tamanho \leftarrow tamanho - 1$
5. **retorne** tamanho

Heap Sort

Algoritmo:

FilhoEsquerdo(iPai)

1. $fEsq \leftarrow 2 * iPai + 1$
2. **retorne** fEsq

Retorna a posição do Filho Esquerdo do nó iPai.

FilhoDireito(iPai)

1. $fDir \leftarrow 2 * iPai + 2$
2. **retorne** fDir

Retorna a posição do Filho Direito do nó iPai.

SelecionaMaximo(vetor[0, ..., n], tamanho)

1. $aux \leftarrow vetor[0]$
2. $vetor[0] \leftarrow vetor[tamanho]$
3. $vetor[tamanho] \leftarrow aux$
4. $tamanho \leftarrow tamanho - 1$
5. **retorne** tamanho

Heap Sort

Algoritmo:

FilhoEsquerdo(iPai)

1. $fEsq \leftarrow 2 * iPai + 1$
2. **retorne** fEsq

Retorna a posição do Filho Esquerdo do nó iPai.

FilhoDireito(iPai)

1. $fDir \leftarrow 2 * iPai + 2$
2. **retorne** fDir

Retorna a posição do Filho Direito do nó iPai.

SelecionaMaximo(vetor[0, ..., n], tamanho)

1. $aux \leftarrow vetor[0]$
2. $vetor[0] \leftarrow vetor[tamanho]$
3. $vetor[tamanho] \leftarrow aux$
4. $tamanho \leftarrow tamanho - 1$
5. **retorne** tamanho

Troca o maior elemento (índice 0) pelo último elemento da árvore heap (índice tamanho).
Decrementa o tamanho, em outras palavras exclui o maior elemento encontrado do heap.

Heap Sort

Algoritmo:

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow \text{FilhoDireito}(ind)$ → Busca o índice do Filho Direito do pai ind.
2. $fEsq \leftarrow \text{FilhoEsquerdo}(ind)$
3. **se** $fEsq < \text{tamanho}$ **e** $\text{vetor}[fEsq] > \text{vetor}[ind]$ **então**
4. $maior \leftarrow fEsq$
5. **senão**
6. $maior \leftarrow ind$
7. **fim-se**
8. **se** $fDir < \text{tamanho}$ **e** $\text{vetor}[fDir] > \text{vetor}[maior]$ **então**
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow \text{vetor}[ind]$
13. $v[ind] \leftarrow v[maior]$
14. $v[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo:

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow \text{FilhoDireito}(ind)$ → Busca o índice do Filho Direito do pai ind.
2. $fEsq \leftarrow \text{FilhoEsquerdo}(ind)$ → Busca o índice do Filho Esquerdo do pai ind.
3. **se** $fEsq < tamanho$ **e** $\text{vetor}[fEsq] > \text{vetor}[ind]$ **então**
4. $maior \leftarrow fEsq$
5. **senão**
6. $maior \leftarrow ind$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $\text{vetor}[fDir] > \text{vetor}[maior]$ **então**
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow \text{vetor}[ind]$
13. $\text{vetor}[ind] \leftarrow \text{vetor}[maior]$
14. $\text{vetor}[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo:

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow \text{FilhoDireito}(ind)$ → Busca o índice do Filho Direito do pai ind.
2. $fEsq \leftarrow \text{FilhoEsquerdo}(ind)$ → Busca o índice do Filho Esquerdo do pai ind.
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** → Verifica se o filho pertence ao heap e se seu valor é maior que o valor do pai.
4. $maior \leftarrow fEsq$
5. **senão**
6. $maior \leftarrow ind$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então**
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow vetor[ind]$
13. $vetor[ind] \leftarrow vetor[maior]$
14. $vetor[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo:

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow \text{FilhoDireito}(ind)$ → Busca o índice do Filho Direito do pai ind.
2. $fEsq \leftarrow \text{FilhoEsquerdo}(ind)$ → Busca o índice do Filho Esquerdo do pai ind.
3. **se** $fEsq < tamanho$ **e** $\text{vetor}[fEsq] > \text{vetor}[ind]$ **então** → Verifica se o filho pertence ao heap e se seu valor é maior que o valor do pai.
4. $maior \leftarrow fEsq$ → Caso verdadeiro, o filho esquerdo é o maior valor na subárvore
5. **senão**
6. $maior \leftarrow ind$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $\text{vetor}[fDir] > \text{vetor}[maior]$ **então**
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow \text{vetor}[ind]$
13. $\text{vetor}[ind] \leftarrow \text{vetor}[maior]$
14. $\text{vetor}[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo:

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow \text{FilhoDireito}(ind)$ → Busca o índice do Filho Direito do pai ind.
2. $fEsq \leftarrow \text{FilhoEsquerdo}(ind)$ → Busca o índice do Filho Esquerdo do pai ind.
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** → Verifica se o filho pertence ao heap e se seu valor é maior que o valor do pai.
4. $maior \leftarrow fEsq$ → Caso verdadeiro, o filho esquerdo é o maior valor na subárvore
5. **senão**
6. $maior \leftarrow ind$ → Caso contrário, o pai ainda é o maior valor na subárvore
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então**
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow vetor[ind]$
13. $vetor[ind] \leftarrow vetor[maior]$
14. $vetor[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo:

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow \text{FilhoDireito}(ind)$ → Busca o índice do Filho Direito do pai ind.
2. $fEsq \leftarrow \text{FilhoEsquerdo}(ind)$ → Busca o índice do Filho Esquerdo do pai ind.
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** → Verifica se o filho pertence ao heap e se seu valor é maior que o valor do pai.
4. $maior \leftarrow fEsq$ → Caso verdadeiro, o filho esquerdo é o maior valor na subárvore
5. **senão**
6. $maior \leftarrow ind$ → Caso contrário, o pai ainda é o maior valor na subárvore
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então** → Verifica se o filho pertence ao heap e se seu valor é maior que o maior valor até agora.
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow vetor[ind]$
13. $vetor[ind] \leftarrow vetor[maior]$
14. $vetor[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo:

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow \text{FilhoDireito}(ind)$ → Busca o índice do Filho Direito do pai ind.
2. $fEsq \leftarrow \text{FilhoEsquerdo}(ind)$ → Busca o índice do Filho Esquerdo do pai ind.
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** → Verifica se o filho pertence ao heap e se seu valor é maior que o valor do pai.
4. $maior \leftarrow fEsq$ → Caso verdadeiro, o filho esquerdo é o maior valor na subárvore
5. **senão**
6. $maior \leftarrow ind$ → Caso contrário, o pai ainda é o maior valor na subárvore
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então** → Verifica se o filho pertence ao heap e se seu valor é maior que o maior valor até agora.
9. $maior \leftarrow fDir$ → Caso verdadeiro, o filho direito é o maior valor na subárvore
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow vetor[ind]$
13. $v[ind] \leftarrow v[maior]$
14. $v[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo:

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow \text{FilhoDireito}(ind)$ → Busca o índice do Filho Direito do pai ind.
 2. $fEsq \leftarrow \text{FilhoEsquerdo}(ind)$ → Busca o índice do Filho Esquerdo do pai ind.
 3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** → Verifica se o filho pertence ao heap e se seu valor é maior que o valor do pai.
 4. $maior \leftarrow fEsq$ → Caso verdadeiro, o filho esquerdo é o maior valor na subárvore
 5. **senão**
 6. $maior \leftarrow ind$ → Caso contrário, o pai ainda é o maior valor na subárvore
 7. **fim-se**
 8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então** → Verifica se o filho pertence ao heap e se seu valor é maior que o maior valor até agora.
 9. $maior \leftarrow fDir$ → Caso verdadeiro, o filho direito é o maior valor na subárvore
 10. **fim-se**
 11. **se** $maior \neq ind$ **então**
 12. $aux \leftarrow vetor[ind]$
 13. $v[ind] \leftarrow v[maior]$
 14. $v[maior] \leftarrow aux$
 15. RestauraHeap(vetor, tamanho, maior)
 16. **fim-se**
- Se for constatado que o maior valor não é o pai da subárvore (índice ind), então troque o pai pelo maior entre os filhos.
E isso pode quebrar o heap da subsubárvore, então por isso, execute RestauraHeap na subsubárvore.

Heap Sort

Algoritmo:

ConstroiHeap(vetor[0, ..., n], tamanho)

1. **para** i **de** $\text{div}(\text{tamanho}, 2) - 1$ **até** 0 **passo** -1 **faça** $\longrightarrow$ Para cada nó pai da árvore.
2. RestauraHeap(vetor, tamanho, i)
3. **fim-para**

Heap Sort

Algoritmo:

ConstroiHeap(vetor[0, ..., n], tamanho)

1. **para** i **de** $\text{div}(\text{tamanho}, 2) - 1$ **até** 0 **passo** -1 **faça** $\rightarrow$ Para cada nó pai da árvore.
2. RestauraHeap(vetor, tamanho, i) $\rightarrow$ Restaure o heap na subárvore.
3. **fim-para**

Heap Sort

Algoritmo:

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho)  Construa um heap binário com o arranjo inicial.
2. **para** i **de** tamanho-1 **até** 0 **passo** -1 **faça**
3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho)
4. RestauraHeap(vetor, tamanho, 0)
5. **fim-para**

Heap Sort

Algoritmo:

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho)  Construa um heap binário com o arranjo inicial.
2. **para** i **de** tamanho-1 **até** 0 **passo** -1 **faça**  Para cada item do arranjo.
3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho)
4. RestauraHeap(vetor, tamanho, 0)
5. **fim-para**

Heap Sort

Algoritmo:

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho) $\longrightarrow$ Construa um heap binário com o arranjo inicial.
2. **para** i **de** tamanho-1 **até** 0 **passo** -1 **faça** $\longrightarrow$ Para cada item do arranjo.
3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow$ Apanhe o maior e exclua-o do heap.
4. RestauraHeap(vetor, tamanho, 0)
5. **fim-para**

Heap Sort

Algoritmo:

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho) $\longrightarrow$ Construa um heap binário com o arranjo inicial.
2. **para** i **de** tamanho-1 **até** 0 **passo** -1 **faça** $\longrightarrow$ Para cada item do arranjo.
3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow$ Apanhe o maior e exclua-o do heap.
4. RestauraHeap(vetor, tamanho, 0) $\longrightarrow$ Reorganize o arranjo para que volte a ser um heap.
5. **fim-para**

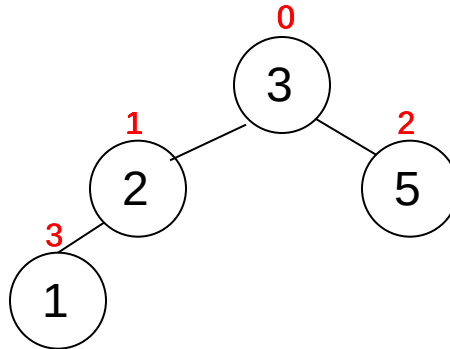
Heap Sort

Teste

vetor =

3	2	5	1
---	---	---	---

tamanho = 4



Funções para auxiliar no próximo slide

Heap Sort

Algoritmo:

FilhoEsquerdo(iPai)

1. $fEsq \leftarrow 2 * iPai + 1$
2. **retorne** fEsq

FilhoDireito(iPai)

1. $fDir \leftarrow 2 * iPai + 2$
2. **retorne** fDir

SelecionaMaximo(vetor[0, ..., n], tamanho)

1. $aux \leftarrow vetor[0]$
2. $vetor[0] \leftarrow vetor[tamanho]$
3. $vetor[tamanho] \leftarrow aux$
4. $tamanho \leftarrow tamanho - 1$
5. **retorne** tamanho

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho)
2. **para** i **de** tamanho-1 **até** 0 **passo** -1 **faça**
3. $tamanho \leftarrow SelecionaMaximo(vetor, tamanho)$
4. RestauraHeap(vetor, tamanho, 0)
5. **fim-para**

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$
2. $fEsq \leftarrow FilhoEsquerdo(ind)$
3. **se** fEsq < tamanho **e** vetor[fEsq] > vetor[ind] **então**
4. $maior \leftarrow fEsq$
5. **senão**
6. $maior \leftarrow ind$
7. **fim-se**
8. **se** fDir < tamanho **e** vetor[fDir] > vetor[maior] **então**
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** maior $\neq$ ind **então**
12. $aux \leftarrow vetor[ind]$
13. $v[ind] \leftarrow v[maior]$
14. $v[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

ConstroiHeap(vetor[0, ..., n], tamanho)

1. **para** i **de** $div(tamanho, 2) - 1$ **até** 0 **passo** -1 **faça**
2. RestauraHeap(vetor, tamanho, i)
3. **fim-para**

Heap Sort

Algoritmo (Análise de Complexidade):

FilhoEsquerdo(iPai) $\left. \begin{array}{l} 1. fEsq \leftarrow 2 * iPai + 1 \\ 2. \textbf{retorne} fEsq \end{array} \right\} \Theta(1)$

FilhoDireito(iPai)
1. fDir $\leftarrow 2 * iPai + 2$
2. **retorne** fDir

SelecionaMaximo(vetor[0, ..., n], tamanho)

1. aux $\leftarrow$ vetor[0]
2. vetor[0] $\leftarrow$ vetor[tamanho]
3. vetor[tamanho] $\leftarrow$ aux
4. tamanho $\leftarrow$ tamanho - 1
5. **retorne** tamanho

Heap Sort

Algoritmo (Análise de Complexidade):

FilhoEsquerdo(iPai) $\left. \begin{array}{l} 1. fEsq \leftarrow 2 * iPai + 1 \\ 2. \textbf{retorne} fEsq \end{array} \right\} \Theta(1)$

FilhoDireito(iPai) $\left. \begin{array}{l} 1. fDir \leftarrow 2 * iPai + 2 \\ 2. \textbf{retorne} fDir \end{array} \right\} \Theta(1)$

SelecionaMaximo(vetor[0, ..., n], tamanho)

1. $aux \leftarrow vetor[0]$
2. $vetor[0] \leftarrow vetor[tamanho]$
3. $vetor[tamanho] \leftarrow aux$
4. $tamanho \leftarrow tamanho - 1$
5. **retorne** tamanho

Heap Sort

Algoritmo (Análise de Complexidade):

FilhoEsquerdo(iPai) $\left. \begin{array}{l} 1. fEsq \leftarrow 2 * iPai + 1 \\ 2. \textbf{retorne} fEsq \end{array} \right\} \Theta(1)$

FilhoDireito(iPai) $\left. \begin{array}{l} 1. fDir \leftarrow 2 * iPai + 2 \\ 2. \textbf{retorne} fDir \end{array} \right\} \Theta(1)$

SelecionaMaximo(vetor[0, ..., n], tamanho) $\left. \begin{array}{l} 1. aux \leftarrow vetor[0] \\ 2. vetor[0] \leftarrow vetor[tamanho] \\ 3. vetor[tamanho] \leftarrow aux \\ 4. tamanho \leftarrow tamanho - 1 \\ 5. \textbf{retorne} tamanho \end{array} \right\} \Theta(1)$

Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow \text{FilhoDireito}(ind)$  $\Theta(1)$.
2. $fEsq \leftarrow \text{FilhoEsquerdo}(ind)$
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então**
4. $maior \leftarrow fEsq$
5. **senão**
6. $maior \leftarrow ind$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então**
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow vetor[ind]$
13. $v[ind] \leftarrow v[maior]$
14. $v[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow \text{FilhoDireito}(ind)$ $\longrightarrow \Theta(1).$
2. $fEsq \leftarrow \text{FilhoEsquerdo}(ind)$ $\longrightarrow \Theta(1).$
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então**
4. $maior \leftarrow fEsq$
5. **senão**
6. $maior \leftarrow ind$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então**
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow vetor[ind]$
13. $v[ind] \leftarrow v[maior]$
14. $v[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$ $\longrightarrow \Theta(1).$
2. $fEsq \leftarrow FilhoEsquerdo(ind)$ $\longrightarrow \Theta(1).$
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** $\longrightarrow \Theta(1).$
4. $maior \leftarrow fEsq$
5. **senão**
6. $maior \leftarrow ind$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então**
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow vetor[ind]$
13. $v[ind] \leftarrow v[maior]$
14. $v[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$ $\rightarrow \Theta(1)$.
2. $fEsq \leftarrow FilhoEsquerdo(ind)$ $\rightarrow \Theta(1)$.
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** $\rightarrow \Theta(1)$.
4. $maior \leftarrow fEsq$ $\rightarrow \Theta(1)$.
5. **senão**
6. $maior \leftarrow ind$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então**
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow vetor[ind]$
13. $vetor[ind] \leftarrow vetor[maior]$
14. $vetor[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$ $\rightarrow \Theta(1).$
2. $fEsq \leftarrow FilhoEsquerdo(ind)$ $\rightarrow \Theta(1).$
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** $\rightarrow \Theta(1).$
4. $maior \leftarrow fEsq$ $\rightarrow \Theta(1).$
5. **senão**
6. $maior \leftarrow ind$ $\rightarrow \Theta(1).$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então**
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow vetor[ind]$
13. $vetor[ind] \leftarrow vetor[maior]$
14. $vetor[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$ $\rightarrow \Theta(1).$
2. $fEsq \leftarrow FilhoEsquerdo(ind)$ $\rightarrow \Theta(1).$
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** $\rightarrow \Theta(1).$
4. $maior \leftarrow fEsq$ $\rightarrow \Theta(1).$
5. **senão**
6. $maior \leftarrow ind$ $\rightarrow \Theta(1).$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então** $\rightarrow \Theta(1).$
9. $maior \leftarrow fDir$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow vetor[ind]$
13. $vetor[ind] \leftarrow vetor[maior]$
14. $vetor[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$ $\rightarrow \Theta(1).$
2. $fEsq \leftarrow FilhoEsquerdo(ind)$ $\rightarrow \Theta(1).$
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** $\rightarrow \Theta(1).$
4. $maior \leftarrow fEsq$ $\rightarrow \Theta(1).$
5. **senão**
6. $maior \leftarrow ind$ $\rightarrow \Theta(1).$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então** $\rightarrow \Theta(1).$
9. $maior \leftarrow fDir$ $\rightarrow \Theta(1).$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow vetor[ind]$
13. $v[ind] \leftarrow v[maior]$
14. $v[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$ $\rightarrow \Theta(1).$
2. $fEsq \leftarrow FilhoEsquerdo(ind)$ $\rightarrow \Theta(1).$
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** $\rightarrow \Theta(1).$
4. $maior \leftarrow fEsq$ $\rightarrow \Theta(1).$
5. **senão**
6. $maior \leftarrow ind$ $\rightarrow \Theta(1).$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então** $\rightarrow \Theta(1).$
9. $maior \leftarrow fDir$ $\rightarrow \Theta(1).$
10. **fim-se**
11. **se** $maior \neq ind$ **então**
12. $aux \leftarrow vetor[ind]$
13. $v[ind] \leftarrow v[maior]$
14. $v[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$ $\rightarrow \Theta(1).$
2. $fEsq \leftarrow FilhoEsquerdo(ind)$ $\rightarrow \Theta(1).$
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** $\rightarrow \Theta(1).$
4. $maior \leftarrow fEsq$ $\rightarrow \Theta(1).$
5. **senão**
6. $maior \leftarrow ind$ $\rightarrow \Theta(1).$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então** $\rightarrow \Theta(1).$
9. $maior \leftarrow fDir$ $\rightarrow \Theta(1).$
10. **fim-se**
11. **se** $maior \neq ind$ **então** $\rightarrow \Theta(1).$
12. $aux \leftarrow vetor[ind]$
13. $v[ind] \leftarrow v[maior]$
14. $v[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$ $\rightarrow \Theta(1).$
2. $fEsq \leftarrow FilhoEsquerdo(ind)$ $\rightarrow \Theta(1).$
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** $\rightarrow \Theta(1).$
4. $maior \leftarrow fEsq$ $\rightarrow \Theta(1).$
5. **senão**
6. $maior \leftarrow ind$ $\rightarrow \Theta(1).$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então** $\rightarrow \Theta(1).$
9. $maior \leftarrow fDir$ $\rightarrow \Theta(1).$
10. **fim-se**
11. **se** $maior \neq ind$ **então** $\rightarrow \Theta(1).$
12. $aux \leftarrow vetor[ind]$ $\rightarrow \Theta(1).$
13. $v[ind] \leftarrow v[maior]$
14. $v[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$ $\rightarrow \Theta(1).$
2. $fEsq \leftarrow FilhoEsquerdo(ind)$ $\rightarrow \Theta(1).$
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** $\rightarrow \Theta(1).$
4. $maior \leftarrow fEsq$ $\rightarrow \Theta(1).$
5. **senão**
6. $maior \leftarrow ind$ $\rightarrow \Theta(1).$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então** $\rightarrow \Theta(1).$
9. $maior \leftarrow fDir$ $\rightarrow \Theta(1).$
10. **fim-se**
11. **se** $maior \neq ind$ **então** $\rightarrow \Theta(1).$
12. $aux \leftarrow vetor[ind]$ $\rightarrow \Theta(1).$
13. $v[ind] \leftarrow v[maior]$ $\rightarrow \Theta(1).$
14. $v[maior] \leftarrow aux$
15. RestauraHeap(vetor, tamanho, maior)
16. **fim-se**

Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$ $\rightarrow \Theta(1).$
2. $fEsq \leftarrow FilhoEsquerdo(ind)$ $\rightarrow \Theta(1).$
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** $\rightarrow \Theta(1).$
4. $maior \leftarrow fEsq$ $\rightarrow \Theta(1).$
5. **senão**
6. $maior \leftarrow ind$ $\rightarrow \Theta(1).$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então** $\rightarrow \Theta(1).$
9. $maior \leftarrow fDir$ $\rightarrow \Theta(1).$
10. **fim-se**
11. **se** $maior \neq ind$ **então** $\rightarrow \Theta(1).$
12. $aux \leftarrow vetor[ind]$ $\rightarrow \Theta(1).$
13. $v[ind] \leftarrow v[maior]$ $\rightarrow \Theta(1).$
14. $v[maior] \leftarrow aux$ $\rightarrow \Theta(1).$
15. RestauraHeap(vetor, tamanho, maior) $\rightarrow ?$
16. **fim-se**

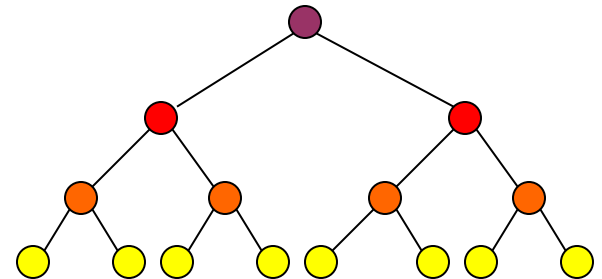
Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$ $\rightarrow \Theta(1)$.
2. $fEsq \leftarrow FilhoEsquerdo(ind)$ $\rightarrow \Theta(1)$.
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** $\rightarrow \Theta(1)$.
4. $maior \leftarrow fEsq$ $\rightarrow \Theta(1)$.
5. **senão**
6. $maior \leftarrow ind$ $\rightarrow \Theta(1)$.
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então** $\rightarrow \Theta(1)$.
9. $maior \leftarrow fDir$ $\rightarrow \Theta(1)$.
10. **fim-se**
11. **se** $maior \neq ind$ **então** $\rightarrow \Theta(1)$.
12. $aux \leftarrow vetor[ind]$ $\rightarrow \Theta(1)$.
13. $v[ind] \leftarrow v[maior]$ $\rightarrow \Theta(1)$.
14. $v[maior] \leftarrow aux$ $\rightarrow \Theta(1)$.
15. RestauraHeap(vetor, tamanho, maior) $\rightarrow ?$
16. **fim-se**

No pior caso, quantas vezes
RestauraHeap é executado?



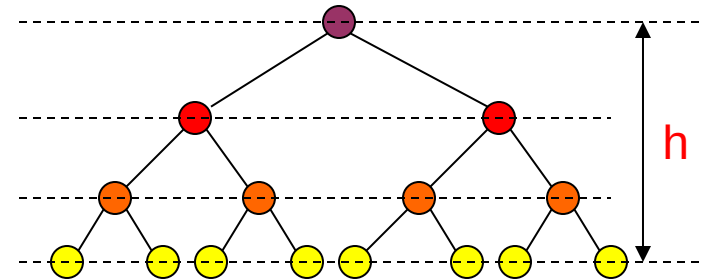
Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow \text{FilhoDireito}(ind)$ $\rightarrow \Theta(1)$.
2. $fEsq \leftarrow \text{FilhoEsquerdo}(ind)$ $\rightarrow \Theta(1)$.
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** $\rightarrow \Theta(1)$.
4. $maior \leftarrow fEsq$ $\rightarrow \Theta(1)$.
5. **senão**
6. $maior \leftarrow ind$ $\rightarrow \Theta(1)$.
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então** $\rightarrow \Theta(1)$.
9. $maior \leftarrow fDir$ $\rightarrow \Theta(1)$.
10. **fim-se**
11. **se** $maior \neq ind$ **então** $\rightarrow \Theta(1)$.
12. $aux \leftarrow vetor[ind]$ $\rightarrow \Theta(1)$.
13. $v[ind] \leftarrow v[maior]$ $\rightarrow \Theta(1)$.
14. $v[maior] \leftarrow aux$ $\rightarrow \Theta(1)$.
15. RestauraHeap(vetor, tamanho, maior) $\rightarrow O(h)$
16. **fim-se**

No pior caso, quantas vezes
RestauraHeap é executado?



Heap Sort

Algoritmo (Análise de Complexidade):

RestauraHeap(vetor[0, ..., n], tamanho, ind)

1. $fDir \leftarrow FilhoDireito(ind)$ $\rightarrow \Theta(1).$
2. $fEsq \leftarrow FilhoEsquerdo(ind)$ $\rightarrow \Theta(1).$
3. **se** $fEsq < tamanho$ **e** $vetor[fEsq] > vetor[ind]$ **então** $\rightarrow \Theta(1).$
4. $maior \leftarrow fEsq$ $\rightarrow \Theta(1).$
5. **senão**
6. $maior \leftarrow ind$ $\rightarrow \Theta(1).$
7. **fim-se**
8. **se** $fDir < tamanho$ **e** $vetor[fDir] > vetor[maior]$ **então** $\rightarrow \Theta(1).$
9. $maior \leftarrow fDir$ $\rightarrow \Theta(1).$
10. **fim-se**
11. **se** $maior \neq ind$ **então** $\rightarrow \Theta(1).$
12. $aux \leftarrow vetor[ind]$ $\rightarrow \Theta(1).$
13. $v[ind] \leftarrow v[maior]$ $\rightarrow \Theta(1).$
14. $v[maior] \leftarrow aux$ $\rightarrow \Theta(1).$
15. RestauraHeap(vetor, tamanho, maior) $\rightarrow O(h)$
16. **fim-se**

$O(h)$

A função RestauraHeap(...) possui consumo de tempo $O(h)$, onde h é a altura da árvore heap binário.

Heap Sort

Algoritmo (Análise de Complexidade):

ConstroiHeap(vetor[0, ..., n], tamanho)

1. **para** i **de** $\text{div}(\text{tamanho}, 2)$ **até** 0 **passo** -1 **faça** $\rightarrow O(n)$
2. RestauraHeap(vetor, tamanho, i)
3. **fim-para**

Heap Sort

Algoritmo (Análise de Complexidade):

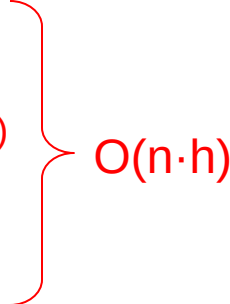
ConstroiHeap(vetor[0, ..., n], tamanho)

1. **para** i **de** $\text{div}(\text{tamanho}, 2)$ **até** 0 **passo** -1 **faça** $\rightarrow O(n)$
2. RestauraHeap(vetor, tamanho, i) $\rightarrow O(h)$
3. **fim-para**

Heap Sort

Algoritmo (Análise de Complexidade):

ConstroiHeap(vetor[0, ..., n], tamanho)

1. **para** i **de** $\text{div}(\text{tamanho}, 2)$ **até** 0 **passo** -1 **faça** $\rightarrow O(n)$
 2. RestauraHeap(vetor, tamanho, i) $\rightarrow O(h)$
 3. **fim-para**
- 

Um arranjo qualquer é transformado em um heap consumindo tempo $O(n \cdot h)$, onde n é o tamanho do arranjo e h a altura da árvore.

Heap Sort

Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho)  $O(n \cdot h)$
2. **para** i **de** tamanho-1 **até** 0 **passo** -1 **faça**
3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho)
4. RestauraHeap(vetor, tamanho, i)
5. **fim-para**

Heap Sort

Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
2. **para** i **de** tamanho-1 **até** 0 **passo** -1 **faça** $\longrightarrow O(n)$
3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho)
4. RestauraHeap(vetor, tamanho, i)
5. **fim-para**

Heap Sort

Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
2. **para** i **de** tamanho-1 **até** 0 **passo** -1 **faça** $\longrightarrow O(n)$
3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
4. RestauraHeap(vetor, tamanho, i)
5. **fim-para**

Heap Sort

Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
2. **para** i **de** tamanho-1 **até** 0 **passo** -1 **faça** $\longrightarrow O(n)$
3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
4. RestauraHeap(vetor, tamanho, i) $\longrightarrow O(h)$
5. **fim-para**

Heap Sort

Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
 2. **para** i **de** tamanho-1 **até** 0 **passo** -1 **faça** $\longrightarrow O(n)$
 3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
 4. RestauraHeap(vetor, tamanho, i) $\longrightarrow O(h)$
 5. **fim-para**
- $\left. \begin{array}{l} \text{2. para } i \text{ de tamanho-1 até 0 passo -1 faça} \\ \text{3. tamanho} \leftarrow \text{SeleccionaMaximo(vetor, tamanho)} \\ \text{4. RestauraHeap(vetor, tamanho, } i \text{)} \end{array} \right\} O(n \cdot h)$

Heap Sort

Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
 2. **para** i de tamanho-1 até 0 **passo** -1 **faça** $\longrightarrow O(n)$
 3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
 4. RestauraHeap(vetor, tamanho, i) $\longrightarrow O(h)$
 5. **fim-para**
- $\left. \begin{array}{l} O(n \cdot h) \\ O(n) \\ \Theta(1) \\ O(h) \end{array} \right\} O(n \cdot h)$ $\left. \begin{array}{l} 2 \cdot O(n \cdot h) \\ = \\ O(n \cdot h) \end{array} \right\}$

HeapSort é capaz de ordenar os n elementos do arranjo com consumo de tempo proporcional a $O(n \cdot h)$, onde n é o número de itens do arranjo e h a altura do heap.

Heap Sort

Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
 2. **para** i de tamanho-1 **até** 0 **passo** -1 **faça** $\longrightarrow O(n)$
 3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
 4. RestauraHeap(vetor, tamanho, i) $\longrightarrow O(h)$
 5. **fim-para**
- $\left. \begin{array}{l} O(n \cdot h) \\ O(n) \\ \Theta(1) \\ O(h) \end{array} \right\} O(n \cdot h)$ $\left. \begin{array}{l} 2 \cdot O(n \cdot h) \\ = \\ O(n \cdot h) \end{array} \right\}$

HeapSort é capaz de ordenar os n elementos do arranjo com consumo de tempo proporcional a $O(n \cdot h)$, onde n é o número de itens do arranjo e h a altura do heap.

É possível escrevermos h em função de n ?

Heap Sort

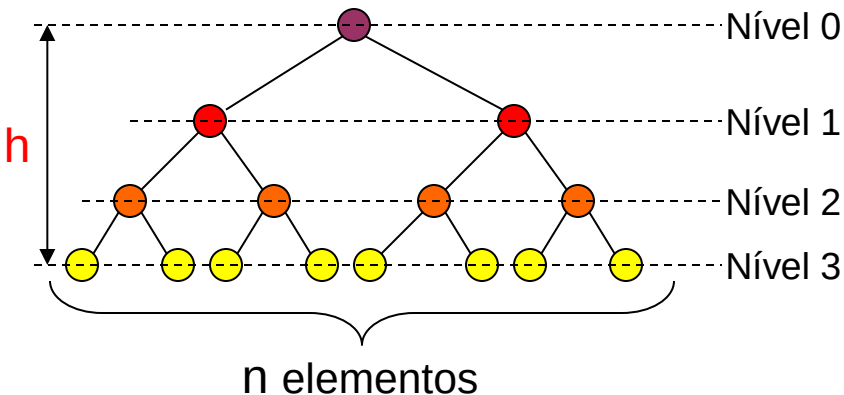
Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
 2. **para** i **de** tamanho-1 **até** 0 **passo** -1 **faça** $\longrightarrow O(n)$
 3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
 4. RestauraHeap(vetor, tamanho, i) $\longrightarrow O(h)$
 5. **fim-para**
- $\left. \begin{array}{l} O(n \cdot h) \\ O(n) \\ \Theta(1) \\ O(h) \end{array} \right\} O(n \cdot h)$ $\left. \begin{array}{l} 2 \cdot O(n \cdot h) \\ = \\ O(n \cdot h) \end{array} \right\}$

HeapSort é capaz de ordenar os n elementos do arranjo com consumo de tempo proporcional a $O(n \cdot h)$, onde n é o número de itens do arranjo e h a altura do heap.

É possível escrevermos h em função de n ?



Heap Sort

Algoritmo (Análise de Complexidade):

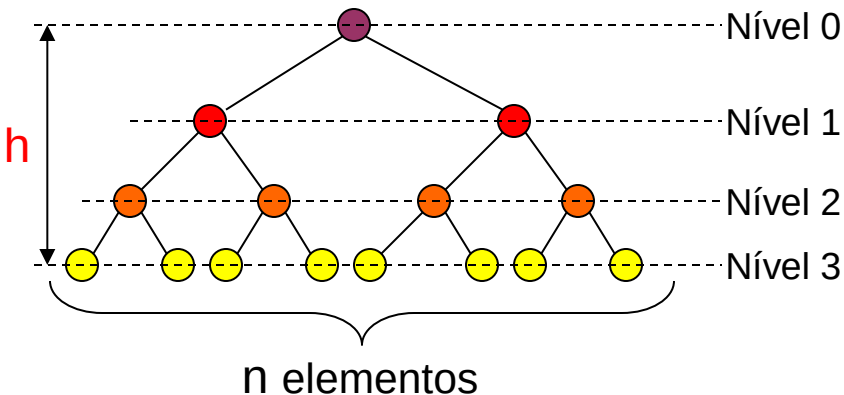
HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
 2. **para** i **de** tamanho-1 **até** 0 **passo** -1 **faça** $\longrightarrow O(n)$
 3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
 4. RestauraHeap(vetor, tamanho, i) $\longrightarrow O(h)$
 5. **fim-para**
- $\left. \begin{array}{l} O(n \cdot h) \\ O(n) \\ \Theta(1) \\ O(h) \end{array} \right\} O(n \cdot h)$ $\left. \begin{array}{l} 2 \cdot O(n \cdot h) \\ = \\ O(n \cdot h) \end{array} \right\}$

HeapSort é capaz de ordenar os n elementos do arranjo com consumo de tempo proporcional a $O(n \cdot h)$, onde n é o número de itens do arranjo e h a altura do heap.

É possível escrevermos h em função de n ?

$$h = \lfloor \log_2 n \rfloor$$



Heap Sort

Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

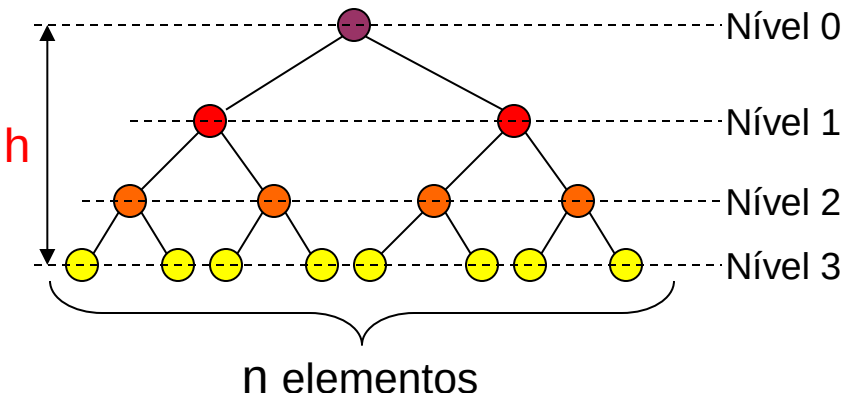
1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
 2. **para** i de tamanho-1 até 0 **passo** -1 **faça** $\longrightarrow O(n)$
 3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
 4. RestauraHeap(vetor, tamanho, i) $\longrightarrow O(h)$
 5. **fim-para**
- $\left. \begin{array}{l} O(n \cdot h) \\ O(n) \\ \Theta(1) \\ O(h) \end{array} \right\} O(n \cdot h)$ $\left. \begin{array}{l} 2 \cdot O(n \cdot h) \\ = \\ O(n \cdot h) \end{array} \right\}$

HeapSort é capaz de ordenar os n elementos do arranjo com consumo de tempo proporcional a $O(n \cdot h)$, onde n é o número de itens do arranjo e h a altura do heap.

É possível escrevermos h em função de n ?

$$h = \lfloor \log_2 n \rfloor$$

n	$\log_2 n$
15	3,91



Heap Sort

Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

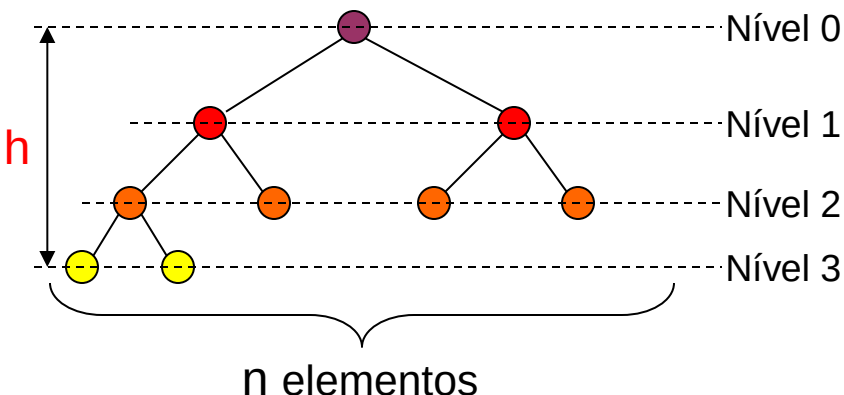
1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
 2. **para** i de tamanho-1 até 0 **passo** -1 **faça** $\longrightarrow O(n)$
 3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
 4. RestauraHeap(vetor, tamanho, i) $\longrightarrow O(h)$
 5. **fim-para**
- $\left. \begin{array}{l} O(n \cdot h) \\ O(n) \\ \Theta(1) \\ O(h) \end{array} \right\} O(n \cdot h) \left\{ \begin{array}{l} 2 \cdot O(n \cdot h) \\ = \\ O(n \cdot h) \end{array} \right.$

HeapSort é capaz de ordenar os n elementos do arranjo com consumo de tempo proporcional a $O(n \cdot h)$, onde n é o número de itens do arranjo e h a altura do heap.

É possível escrevermos h em função de n ?

$$h = \lfloor \log_2 n \rfloor$$

n	$\log_2 n$
15	3,91
9	3,17



Heap Sort

Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

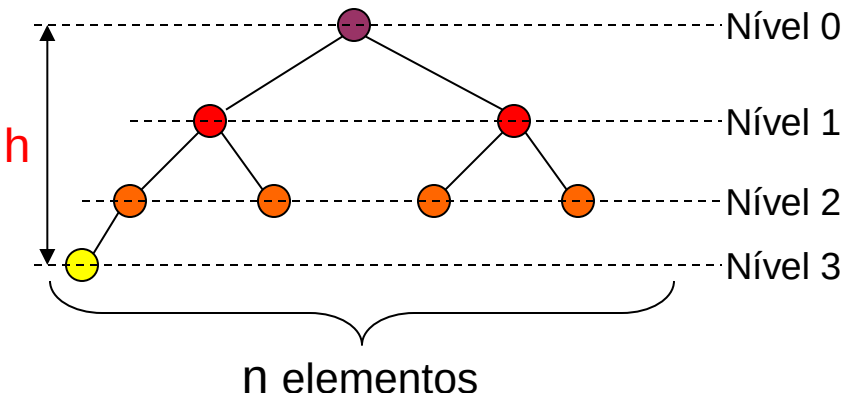
1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
 2. **para** i de tamanho-1 até 0 **passo** -1 **faça** $\longrightarrow O(n)$
 3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
 4. RestauraHeap(vetor, tamanho, i) $\longrightarrow O(h)$
 5. **fim-para**
- $\left. \begin{array}{l} O(n \cdot h) \\ O(n) \\ \Theta(1) \\ O(h) \end{array} \right\} O(n \cdot h)$ $\left. \begin{array}{l} 2 \cdot O(n \cdot h) \\ = \\ O(n \cdot h) \end{array} \right\}$

HeapSort é capaz de ordenar os n elementos do arranjo com consumo de tempo proporcional a $O(n \cdot h)$, onde n é o número de itens do arranjo e h a altura do heap.

É possível escrevermos h em função de n ?

$$h = \lfloor \log_2 n \rfloor$$

n	$\log_2 n$
15	3,91
9	3,17
8	3



Heap Sort

Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

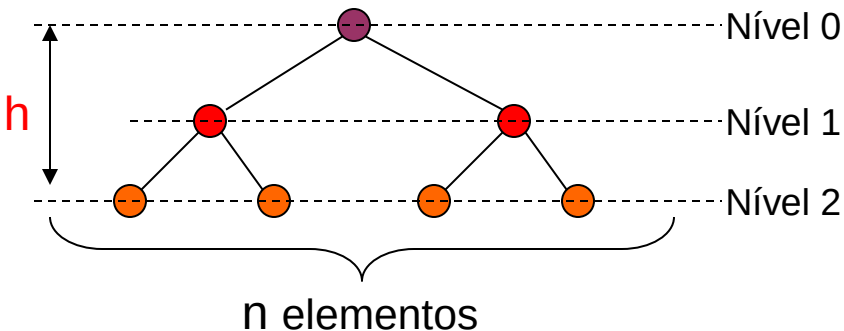
1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
 2. **para** i de tamanho-1 até 0 **passo** -1 **faça** $\longrightarrow O(n)$
 3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
 4. RestauraHeap(vetor, tamanho, i) $\longrightarrow O(h)$
 5. **fim-para**
- $\left. \begin{array}{l} O(n \cdot h) \\ O(n) \\ \Theta(1) \\ O(h) \end{array} \right\} O(n \cdot h)$ $\left. \begin{array}{l} 2 \cdot O(n \cdot h) \\ = \\ O(n \cdot h) \end{array} \right\}$

HeapSort é capaz de ordenar os n elementos do arranjo com consumo de tempo proporcional a $O(n \cdot h)$, onde n é o número de itens do arranjo e h a altura do heap.

É possível escrevermos h em função de n ?

$$h = \lfloor \log_2 n \rfloor$$

n	$\log_2 n$
15	3,91
9	3,17
8	3
7	2,81



Heap Sort

Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

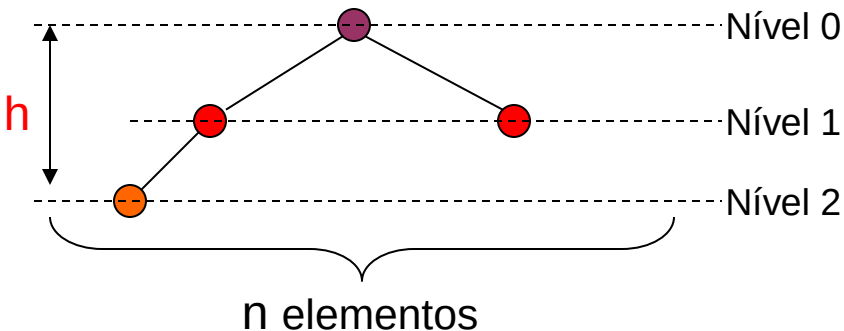
1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
 2. **para** i **de** tamanho-1 **até** 0 **passo** -1 **faça** $\longrightarrow O(n)$
 3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
 4. RestauraHeap(vetor, tamanho, i) $\longrightarrow O(h)$
 5. **fim-para**
- $\left. \begin{array}{l} O(n \cdot h) \\ O(n) \\ \Theta(1) \\ O(h) \end{array} \right\} O(n \cdot h) \left\{ \begin{array}{l} 2 \cdot O(n \cdot h) \\ = \\ O(n \cdot h) \end{array} \right.$

HeapSort é capaz de ordenar os n elementos do arranjo com consumo de tempo proporcional a $O(n \cdot h)$, onde n é o número de itens do arranjo e h a altura do heap.

É possível escrevermos h em função de n ?

$$h = \lfloor \log_2 n \rfloor$$

n	$\log_2 n$
15	3,91
9	3,17
8	3
7	2,81
4	2



Heap Sort

Algoritmo (Análise de Complexidade):

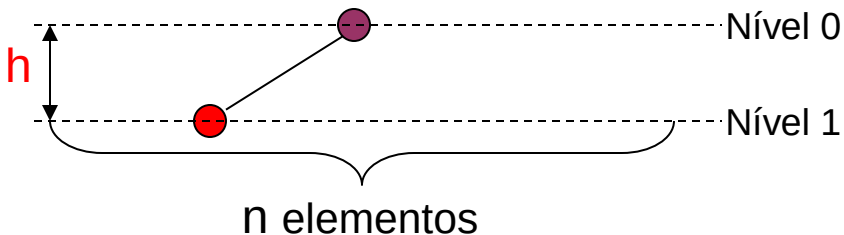
HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
 2. **para** i de tamanho-1 até 0 **passo** -1 **faça** $\longrightarrow O(n)$
 3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
 4. RestauraHeap(vetor, tamanho, i) $\longrightarrow O(h)$
 5. **fim-para**
- $\left. \begin{array}{l} O(n \cdot h) \\ O(n) \\ \Theta(1) \\ O(h) \end{array} \right\} O(n \cdot h) \left\{ \begin{array}{l} 2 \cdot O(n \cdot h) \\ = \\ O(n \cdot h) \end{array} \right.$

HeapSort é capaz de ordenar os n elementos do arranjo com consumo de tempo proporcional a $O(n \cdot h)$, onde n é o número de itens do arranjo e h a altura do heap.

É possível escrevermos h em função de n ?

$$h = \lfloor \log_2 n \rfloor$$



n	$\log_2 n$
15	3,91
9	3,17
8	3
7	2,81
4	2
2	1

Heap Sort

Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
 2. **para** i de tamanho-1 até 0 **passo** -1 **faça** $\longrightarrow O(n)$
 3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
 4. RestauraHeap(vetor, tamanho, i) $\longrightarrow O(h)$
 5. **fim-para**
- $\left. \begin{array}{l} O(n \cdot h) \\ O(n) \\ \Theta(1) \\ O(h) \end{array} \right\} O(n \cdot h)$ $\left. \begin{array}{l} 2 \cdot O(n \cdot h) \\ = \\ O(n \cdot h) \end{array} \right\}$

HeapSort é capaz de ordenar os n elementos do arranjo com consumo de tempo proporcional a $O(n \cdot h)$, onde n é o número de itens do arranjo e h a altura do heap.

É possível escrevermos h em função de n ?

Sim!!! $h = \lfloor \log_2 n \rfloor$

Heap Sort

Algoritmo (Análise de Complexidade):

HeapSort(vetor[0, ..., n], tamanho)

1. ConstroiHeap(vetor, tamanho) $\longrightarrow O(n \cdot h)$
 2. **para** i de tamanho-1 até 0 **passo** -1 **faça** $\longrightarrow O(n)$
 3. tamanho $\leftarrow$ SeleccionaMaximo(vetor, tamanho) $\longrightarrow \Theta(1)$
 4. RestauraHeap(vetor, tamanho, i) $\longrightarrow O(h)$
 5. **fim-para**
- $\left. \begin{array}{l} O(n \cdot h) \\ O(n) \\ \Theta(1) \\ O(h) \end{array} \right\} O(n \cdot h)$ $\left. \begin{array}{l} 2 \cdot O(n \cdot h) \\ = \\ O(n \cdot h) \end{array} \right\}$

HeapSort é capaz de ordenar os n elementos do arranjo com consumo de tempo proporcional a $O(n \cdot h)$, onde n é o número de itens do arranjo e h a altura do heap.

É possível escrevermos h em função de n ?

Sim!!! $h = \lfloor \log_2 n \rfloor$

Portanto, conclui-se que HeapSort é capaz de ordenar os n elementos do arranjo com consumo de tempo proporcional a $O(n \cdot \log_2 n)$, onde n é o número de itens do arranjo.

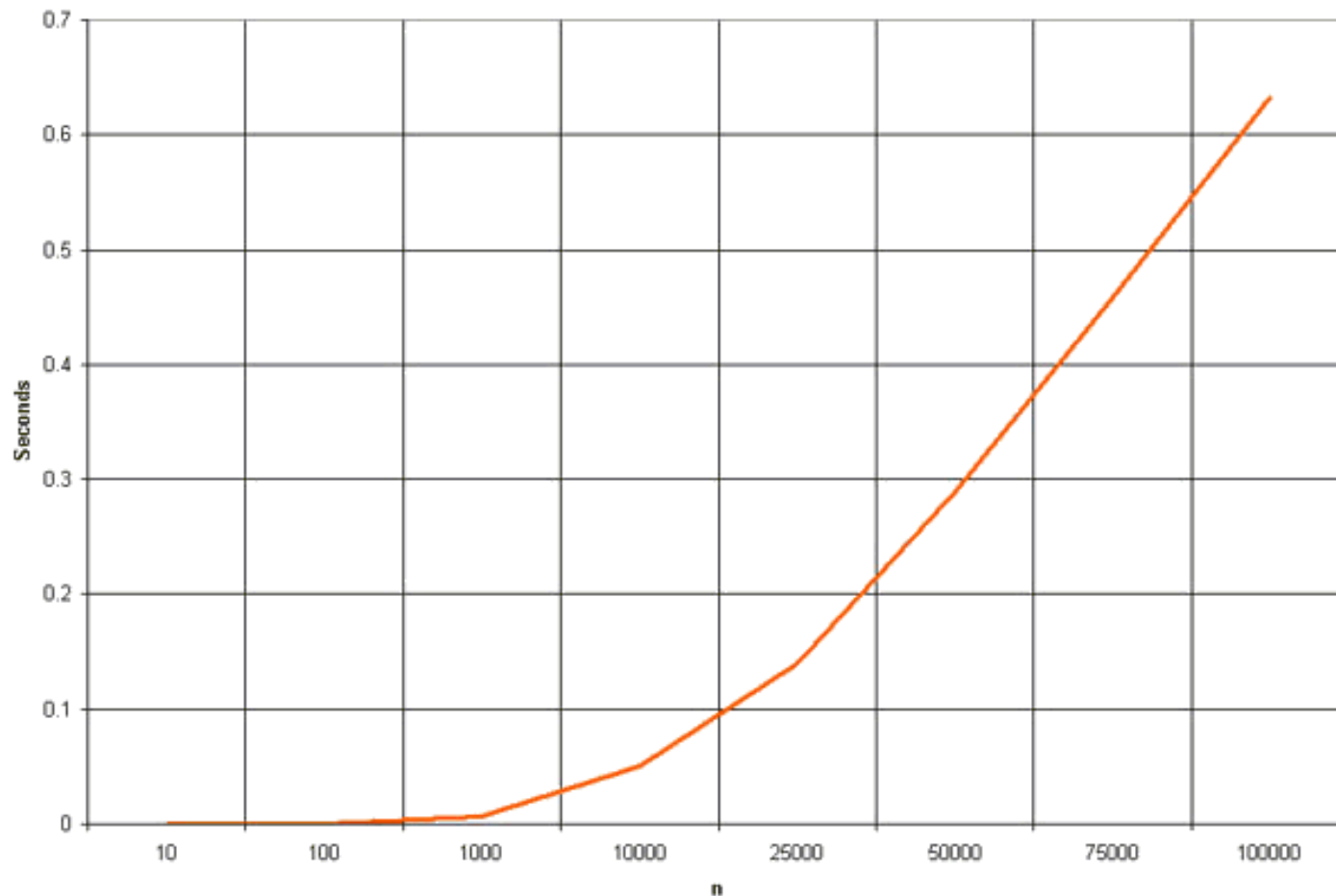
Heap Sort

Algoritmo (Análise de Complexidade):

Procedimento	Consumo no pior caso
FilhoEsquerdo	$\Theta(1)$
FilhoDireito	$\Theta(1)$
SelecionaMaximo	$\Theta(1)$
RestauraHeap	$O(\log_2 n)$
ConstroiHeap	$O(n \cdot \log_2 n)$
HeapSort	$O(n \cdot \log_2 n)$

Heap Sort

- Análise Empírica:



Algoritmos Clássicos de Ordenação

- Quadro Comparativo da complexidade dos algoritmos de ordenação (método de comparação)

Algoritmo	Temporal			Espacial
	Melhor	Médio	Pior	
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$\Theta(n \cdot \log_2 n)$	$\Theta(n \cdot \log_2 n)$	$\Theta(n \cdot \log_2 n)$	$\Theta(n)$
Quick Sort	$\Theta(n \cdot \log_2 n)$	$\Theta(n \cdot \log_2 n)$	$\Theta(n^2)$	$O(\log_2 n)$
Heap Sort	$O(n \cdot \log_2 n)$	$\Theta(n \cdot \log_2 n)$	$O(n \cdot \log_2 n)$	$O(1)$

— Estável

— Não Estável